Process Synchronization: Background

Producer

```
while (true) {
     /* produce an item in nextProduced */
     while (counter == BUFFER_SIZE)
        ; /* do nothing */
     buffer[in] = nextProduced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Consumer

```
while (true) {
     while (counter == 0)
        ; /* do nothing */
     nextConsumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;
     /* consume the item in nextConsumed */
}
```

- Both, the producer and consumer routines are correct separately.
- They may not function correctly when executed concurrently.

Process Synchronization: Background

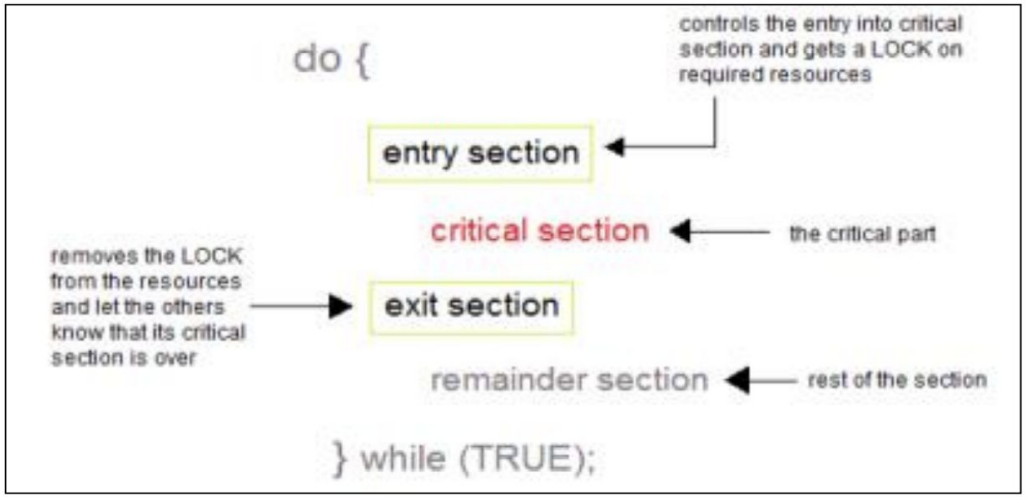
- An incorrect state may arrive because both processes are allowed to manipulate the variable counter concurrently.
- A situation, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a race condition.
- In our example, to guard against the race condition, we need to ensure that only one process at a time can manipulate the variable **counter**.
- To make such a guarantee, the processes must be synchronized in some way.

The Critical-Section Problem

- Consider a system consisting of *n* processes $\{P_0, P_1, \ldots, P_{n-1}\}$.
- Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, etc.
- When one process is executing in its critical section, no other process is to be allowed to execute in its critical section.
  - No two processes are executing in their critical sections at the same time.

The Critical-Section Problem (contd…)

- The solution of critical-section problem involves design of a protocol that the processes can use to cooperate.



- Each process must request permission to enter its critical section.
- The section of code implementing this request is the entry section.
- The critical section may be followed by an exit section.
- The remaining code is the remainder section.

Solution of Critical-Section Problem

- A solution to the critical-section problem must satisfy the following three requirements:
  - Mutual Exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
  - Progress - If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections should decide which will enter its critical section next, in a finite time.
  - Bounded Waiting - After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted. So after the limit is reached, system must grant the process permission to get into its critical section.

Critical Section Problem: Algorithm 1

This algorithm works only for two processes.

Given: int turn; (Shared Variable)

Process P_i

```
turn = i;
do{
     while(turn!=i); <-- ENTRY SECTION
       CRITICAL SECTION
     turn = j; <-- EXIT SECTION
       REMAINDER SECTION
}while(TRUE);
```

Critical Section Problem: Algorithm 1 (contd…)

Given: int turn; (Shared Variable)

Process P_i

```
turn = i;
do{
    while(turn!=i);
      CRITICAL SECTION
    turn = j;
      REMAINDER SECTION
}while(TRUE);
```

Process P_j

```
turn = j;
do{
    while(turn!=j);
      CRITICAL SECTION
    turn = i;
      REMAINDER SECTION
}while(TRUE);
```

This algorithm does not satisfy the progress requirement because there is strict alternation between the processes.

Critical Section Problem: Algorithm 2

This algorithm also works only for two processes.

Given: boolean flag[2]; flag[0] = FALSE; flag[1] = FALSE;

Process P_i

```
do{
    flag[i] = TRUE;
    while(flag[j]); <-- ENTRY SECTION
      CRITICAL SECTION
    flag[i] = FALSE; <-- EXIT SECTION
      REMAINDER SECTION
}while(TRUE);
```

Critical Section Problem: Algorithm 2 (contd…)

Given: boolean flag[2]; flag[0] = FALSE; flag[1] = FALSE;

Process P_i

```
do{
     flag[i] = TRUE;
     while(flag[j]);
       CRITICAL SECTION
     flag[i] = FALSE;
       REMAINDER SECTION
}while(TRUE);
```

Process P_j

```
do{
     flag[j] = TRUE;
     while(flag[i]);
       CRITICAL SECTION
     flag[j] = FALSE;
       REMAINDER SECTION
}while(TRUE);
```

This algorithm can fail the **progress** requirement if both processes set their flags to true and then both execute the while loop.

Critical Section Problem: Peterson's Solution

- Peterson's Solution is a classical software based solution to the critical section problem.
  - Peterson's algorithm is used to synchronize two processes.
  - Peterson's solution requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

  - Variable turn indicates whose turn it is to enter its critical section.
    - If turn == i, then process P_i is allowed to execute in its critical section.
  - Flag array is used to indicate if a process wants to enter its critical section.
    - If flag[i] == true, it indicates that P_i wants to enter its critical section.

Structure of Process Pi in Peterson's Solution

```
do {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] && turn == j);

        critical section

    flag[i] = FALSE;

        remainder section

} while (TRUE);
```

Structure of Process Pi and Pj

Given: boolean flag[2]; int turn; flag[0]=false; flag[1]=false;

Process P_i

```
do {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] == TRUE && turn == j);
        CRITICAL SECTION
    flag[i] = FALSE
        REMAINDER SECTION
}while(TRUE);
```

Process P_j

```
do {
    flag[j] = TRUE;
    turn = i;
    while (flag[i] == TRUE && turn == i);
        CRITICAL SECTION
    flag[j] = FALSE
        REMAINDER SECTION
}while(TRUE);
```

- Peterson's algorithm satisfy the three requirements (mutual exclusive, progress, bounded waiting) of solution of critical section problem.

Limitations of Peterson's Solution

- **Works only for TWO Processes.**
- **Busy Waiting**
  - Busy waiting, also known as spinning, or busy looping is a process synchronization technique in which a process/task waits and constantly checks for a condition to be satisfied before proceeding with its execution.

Waiting Approaches

- There are two general approaches to waiting in operating systems:
  - Busy Waiting - A process/task can continuously check for the condition to be satisfied while consuming the processor.
  - Sleeping (Blocked Waiting or Sleep Waiting) - A process can wait without consuming the processor. In such a case, the process/task is alerted or awakened when the condition is satisfied.

Busy Waiting

- Busy looping is usually used to achieve mutual exclusion in operating systems.
- Busy waiting can be inefficient because the looping procedure is a waste of computer resources
- Although inefficient, busy waiting can be beneficial in mutual exclusion if the waiting time is short and insignificant.
- Additionally, busy waiting is quick and simple to understand and implement.

- A workaround solution for the inefficiency of busy waiting that is implemented in most operating systems is the use of a delay function.

Sleep Waiting

- In this case, the process/task is alerted or awakened when the condition is satisfied.
- A delay function (sleep system call) places the process involved in busy waiting into an inactive state for a specified amount of time.
  - In this case, resources are not wasted as the process is “asleep”.
  - After the sleep time has elapsed, the process is awakened to continue its execution.
  - If the condition is still not satisfied, the sleep time is incremented until the condition can be satisfied.

Bakery Algorithm

- Bakery Algorithm is a critical section solution for **n** processes.
- Each process, wanting to enter critical section, gets a token number.
  - The token numbering scheme always generates numbers in increasing order of enumeration; i.e., 1, 2, 3, 3, 4, 5, …
- A process with lowest token number will enter the critical section.
  - The algorithm preserves the first come first serve property.
- If two processes have same token number, the process with lower process id (PId) will enter the critical section.
- The token number of process is set to 0 when it finishes execution.

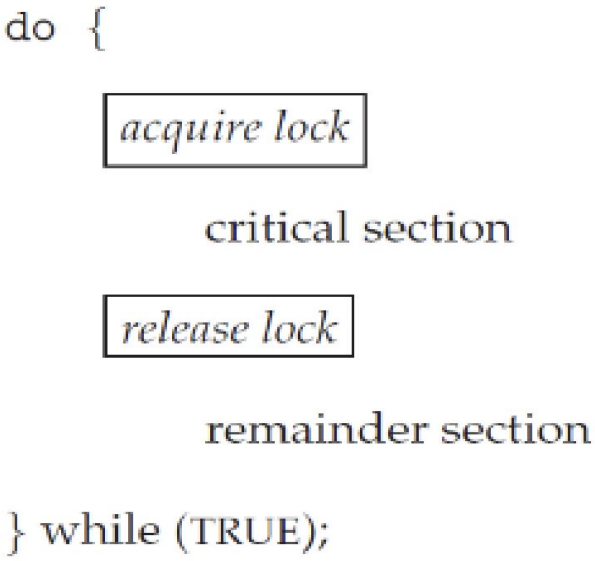
Bakery Algorithm: Structure of Process Pi

```
boolean choosing[N] = {FALSE, ..., FALSE};
int number[N] = {0, ..., 0};
```

Process P_i

```
do {
    choosing[i] = TRUE;
    number[i] = MAX(number[0], number[1], ..., number[n-1]) + 1;
    choosing[i] = FALSE;
    for (j = 0; j < n; j++) {
        while (choosing[j]);
        while ((number[j] != 0) && ((number[j],j) < (number[i],i));
    }
    CRITICAL SECTION
    number[i] = 0;
    REMAINDER SECTION
}while(TRUE);
```

Solution using Lock




```
do {
    while (Lock != 0);
    Lock = 1;
    CRITICAL SECTION
    Lock = 0;
    REMAINDER SECTION
}while(TRUE);
```

Semaphore

- **In 1965, proposed by Dijkstra,** Semaphore is a mechanism that can be used to provide synchronization of tasks (to solve critical-section problem).
- A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait() and signal().
- The initial value of S depends on how many processes are allowed in CS.

wait()

```
wait(S)
{
    while(S <= 0);
    S--;
}
```

The testing of the integer value of S (S ≤ 0), as well as its possible modification (S--), must be executed without interruption.

signal()

```
signal(S)
{
    S++;
}
```

Types of Semaphores

- **Binary Semaphore**
  - It is a special form of semaphore used for implementing mutual exclusion, hence it is often called a Mutex Lock.
  - A binary semaphore is initialized to 1 and only takes the values 0 and 1 during execution of a program.
  - It can be used to deal with the critical-section problem for multiple processes.

Types of Semaphores

- Counting Semaphore
  - Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.
  - It is initialized to the number of resources available.
  - Each process that wishes to use a resource performs a wait() operation on the semaphore (decrementing the count).
  - When a process releases a resource, it performs a signal() operation (incrementing the count).
  - When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

Implementation of Semaphore

Mutual-Exclusion Implementation with Semaphores

```
do{
     wait(S);
     CRITICAL SECTION
     signal(S);
     REMAINDER SECTION
}while(TRUE)
```

```
wait(S){
   while(S <= 0);
   S--;
}
```

```
signal(S){
    S++;
}
```

- The main disadvantage of the semaphore definition given here is that it requires busy waiting.
  - While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.

Spinlock

- Busy waiting wastes CPU cycles that some other process might be able to use productively.
- The semaphore having busy waiting mechanism is also called a spinlock because the process "spins" while waiting for the lock.
- Spinlock mechanism has an advantage in that no context switch is required when a process waits on a lock
  - When locks are expected to be held for short times, spinlocks are useful.
  - Spinlocks are often employed on multiprocessor systems where one thread can "spin" on one processor while another thread performs its critical section on another processor.

Solution of Spinlock (Busy Waiting)

- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() semaphore operations.
- When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait.
  - However, rather than engaging in busy waiting, the process can block itself.
  - The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then control is transferred to the CPU scheduler, which selects another process to execute.

Solution of Spinlock (Busy Waiting)

- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() semaphore operations.

```
typedef struct {
     int value;
     struct process *list;
} semaphore;
```

The block() operation suspends the process that invokes it. The wakeup(P) operation resumes the execution of a blocked process P. These two operations are provided by the operating system as basic system calls.

```
wait(semaphore *S) {
          S->value--;
          if (S->value < 0) {
               add this process to S->list;
               block();
          }
}

signal(semaphore *S) {
          S->value++;
          if (S->value <= 0) {
               remove a process P from S->list;
               wakeup(P);
          }
}
```

Solution of Spinlock (Busy Waiting)

- In the solution of busy waiting, semaphore values may be negative.
- Semaphore values are never negative under the classical definition of semaphores with busy waiting.
- If a semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.
- The list of waiting processes can be easily implemented by a link field in each process control block (PCB).
- To ensure bounded waiting, a FIFO queue or any other queuing strategy may be used.

Limitations of Semaphores

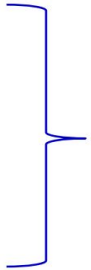
- Deadlocks
  - Consider a system consisting of two processes, P0 and P1, each accessing two semaphores, S and Q, set to the value 1:

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

- Suppose that P0 executes wait(S) and then P1 executes wait(Q).
- When P0 executes wait(Q), it must wait until P1 executes signal(Q).
- Similarly, when P1 executes wait(S), it must wait until P0 executes signal(S).
- Since these signal() operations cannot be executed, P0 and P1 are deadlocked.

Classic Problems of Synchronization

- Bounded-Buffer Problem
- Readers–Writers Problem
- Dining-Philosophers Problem



Solution using Semaphore

Bounded-Buffer Problem

Producer

```
while (true) {
     /* produce an item in nextProduced */
     while (counter == BUFFER_SIZE)
        ; /* do nothing */
     buffer[in] = nextProduced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Consumer

```
while (true) {
     while (counter == 0)
        ; /* do nothing */
     nextConsumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;
     /* consume the item in nextConsumed */
}
```

- The bounded-buffer problem is also known as producer-consumer problem.
- Both, the producer and consumer routines are correct separately.
- They may not function correctly when executed concurrently.

Bounded-Buffer Problem - Solution

- Assume,
  - Buffer Size = n slots
  - semaphore mutex (for mutual exclusion)
    mutex = 1
  - semaphore empty (number of empty slots in buffer)
    empty = n
  - semaphore full (number of full slots in buffer)
    full = 0

Bounded-Buffer Problem - Solution

Producer Process

```
do {
    . . .
   // produce an item in nextp
    . . .
   wait(empty);
   wait(mutex);
    . . .
   // add nextp to buffer
    . . .
   signal(mutex);
   signal(full);
} while (TRUE);
```

Consumer Process

```
do {
   wait(full);
   wait(mutex);
    . . .
   // remove an item from buffer to nextc
    . . .
   signal(mutex);
   signal(empty);
    . . .
   // consume the item in nextc
    . . .
} while (TRUE);
```

Readers–Writers Problem

- Consider a situation where a shared resource (such as file or dataset) is to be accessed by multiple concurrent processes.
- Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database.
- We distinguish between these two types of processes - reader and writer.
  - If two readers access the shared data simultaneously, no adverse effects will result.
  - If a writer and some other process (either a reader or a writer) access the database simultaneously, inconsistency may occur.
- This synchronization problem is referred to as the readers–writers problem.

Readers–Writers Problem - Solution

- Assume,
  - int readcount = 0

    The readcount variable keeps track of how many processes are currently reading the object.

  - semaphore mutex

    mutext is used for mutual exclusion when the variable readcount is updated.

    mutex = 1

  - semaphore wrt

    wrt functions as a mutual-exclusion semaphore for the writers.

    wrt = 1

Readers–Writers Problem - Solution

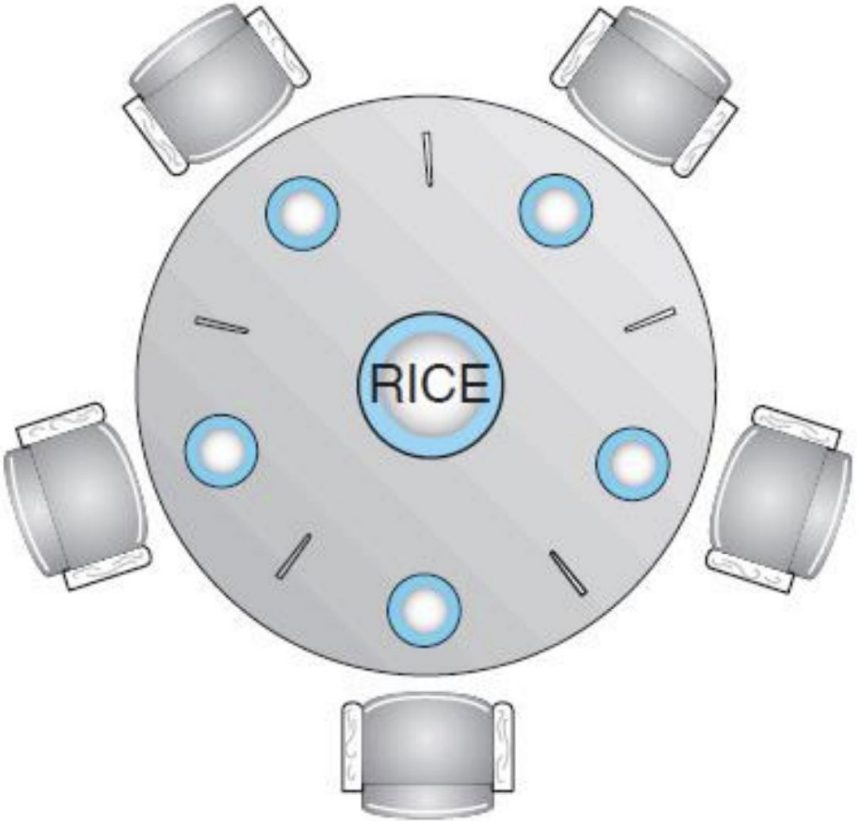
Write Process(s)

```
do {
   wait(wrt);
      . . .
   // writing is performed
      . . .
   signal(wrt);
} while (TRUE);
```

Reader Process(s)

```
do {
   wait(mutex);
   readcount++;
   if (readcount == 1)
      wait(wrt);
   signal(mutex);
      . . .
   // reading is performed
      . . .
   wait(mutex);
   readcount--;
   if (readcount == 0)
      signal(wrt);
   signal(mutex);
} while (TRUE);
```

Dining-Philosophers Problem



Dining-Philosophers Problem

- Consider a situation where five philosophers share a circular table surrounded by five chairs, each belonging to one philosopher.
- They are in thinking-eating cycle.
- In the center of the table is a bowl of rice, and the table is laid with five single chopsticks.
- A philosopher may pick up only one chopstick at a time. (Obviously, he/she cannot pick up a chopstick that is already in the hand of a neighbor).
- From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to his/her (the chopsticks that are between his/her and his/her left and right neighbors).
- The problem is how to choose two chopsticks (one his/her own and one of other)

Dining-Philosophers Problem - Solution

- The Dining-Philosopher Problem is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner.
- The problem can be solved by representing each chopstick with a semaphore.
  - A philosopher tries to grab a chopstick by executing a **wait()** operation on that semaphore.
  - He/she releases his/her chopsticks by executing the **signal()** operation on the appropriate semaphores.
  - **semaphore chopstick[5] = 1;**

Dining-Philosophers Problem - Solution

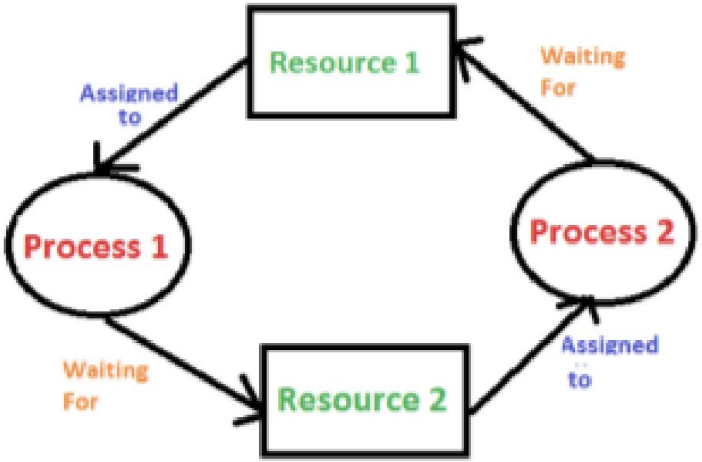
```
do {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   // eat
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   // think
      . . .
} while (TRUE);
```

Limitations of Solution

- Although this solution guarantees that no two neighbors are eating simultaneously, but it could create a deadlock.
  - Suppose that all five philosophers become hungry simultaneously and each grabs her left chopstick.
  - All the elements of chopstick will now be equal to 0.
  - When each philosopher tries to grab her right chopstick, she will be delayed forever.
- Possible Solutions:
  - Allow at most four philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up her chopsticks only if both chopsticks are available.

Deadlock

- In a multiprogramming environment, several processes may compete for a finite number of resources.
- A process requests resources; if the resources are not available at that time, the process enters a waiting state.
- Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes.
- This situation is called a deadlock.



Deadlock (contd…)

- Deadlock can be defined as the permanent blocking of a set of processes that either compete for system resources or communicate with each other.

- A set of processes is deadlocked when each process in the set is blocked awaiting an event (typically the freeing up of some requested resource) that can only be triggered by another blocked process in the set.

Deadlock Characterization

- Necessary and Sufficient Conditions - A deadlock situation arises if the following four conditions hold simultaneously in a system:
  - **Mutual Exclusion** - Only one process at a time can use the resource.
  - **Hold and Wait** - A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
  - **No-Preemption** - Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
  - **Circular Wait** - A set {P0, P1, ..., Pn} of waiting processes must exist such that P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, ..., Pn−1 is waiting for a resource held by Pn, and Pn is waiting for a resource held by P0.

Methods for Handling Deadlocks

- Deadlock Ignorance
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection and Recovery

Deadlock Ignorance

- We can ignore the problem altogether and pretend that deadlocks never occur in the system.

- This approach is used by most operating systems, including UNIX and Windows; it is then up to the application developer to write programs that handle deadlocks.

  - There is always a tradeoff between Correctness and Performance. The operating systems like Windows and Linux mainly focus upon performance. However, the performance of the system decreases if it uses deadlock handling mechanism all the time if deadlock happens 1 out of 100 times then it is completely unnecessary to use the deadlock handling mechanism all the time.

Deadlock Prevention

- If we can be able to violate one of the four necessary conditions and don't let them occur together then we can prevent the deadlock.
  - Mutual Exclusion
    - **If we can be able to violate resources behaving in the mutually exclusive manner then the deadlock can be prevented.**
    - For a device like printer, **Spooling can work** - a memory is associated with the printer which stores jobs from each of the process into it. The printer collects all the jobs and print each one of them according to FCFS. By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing.
    - Although, **Spooling** can be an effective approach to violate mutual exclusion but it suffers from two kinds of problems: **(1)** This cannot be applied to every resource, and **(2)** After some point of time, there may arise a race condition between the processes to get space in that spool.
    - We cannot force a resource to be used by more than one process at the same time. Therefore, we cannot violate mutual exclusion for a process practically.

Deadlock Prevention (contd…)

- Hold and Wait
  - **The hold-and-wait condition can be prevented by requiring that a process request all of its required resources at one time and blocking the process until all requests can be granted simultaneously.**
  - This approach is inefficient in two ways:
    1) First, a process may be held up for a long time waiting for all of its resource requests to be filled, when in fact it could have proceeded with only some of the resources.
    2) Resources allocated to a process may remain unused for a considerable period, during which time they are denied to other processes.
  - Another problem is that a process may not know in advance all of the resources that it will require.

Deadlock Prevention (contd…)

- No-Preemption
  - **If a process holding certain resources is denied a further request, that process must release its original resources and, if necessary, request them again together with the additional resource.**
  - **If a process requests a resource that is currently held by another process, the operating system may preempt the second process and require it to release its resources.**
  - This approach is practical only when applied to resources whose state can be easily saved and restored later, as is the case with a processor.

Deadlock Prevention (contd…)

- Circular Wait
  - **The circular-wait condition can be prevented by defining a linear ordering of resource types.**
  - If a process has been allocated resources of type R, then it may subsequently request only those resources of types following R in the ordering.
  - This approach may be inefficient, slowing down processes and denying resource access unnecessarily.

Deadlock Avoidance

- Deadlock avoidance allows the three necessary conditions but makes judicious choices to assure that the deadlock point is never reached.
- With deadlock avoidance, a decision is made dynamically whether the current resource allocation request will, if granted, potentially lead to a deadlock.
- Deadlock avoidance thus requires knowledge of future process resource requests.
- **Approaches to deadlock avoidance:**
  - Process Initiation Denial
  - Resource Allocation Denial

Process Initiation Denial

Consider a system with n processes and m different types of resources.

[Resource = $\mathbf{R} = (R_1, R_2, \ldots, R_m)$	total amount of each resource in the system
Available = $\mathbf{V} = (V_1, V_2, \ldots, V_m)$	total amount of each resource not allocated to any process
Claim = $\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix}$	C_{ij} = requirement of process i for resource j
Allocation = $\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}$	A_{ij} = current allocation to process i of resource j

Resource Allocation Denial (Banker's Algorithm)

- Why Banker's algorithm is named so?
  - Banker's algorithm is named so because it is used in banking system to check whether loan can be sanctioned to a person or not.
  - The customers who wish to borrow money corresponds to processes and the money to be borrowed corresponds to resources.
  - Suppose there are **n** number of account holders in a bank and the total sum of their money is **S**. If a person applies for a loan then the bank first subtracts the loan amount from the total money that bank has and if the remaining amount is greater than **S** then only the loan is sanctioned. It is done because if all the account holders comes to withdraw their money then the bank can easily do it.
  - In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in safe state always.

Banker's Algorithm

- Consider a system with a fixed number of processes and a fixed number of resources.
- At any time a process may have zero or more resources allocated to it.
- The state of the system reflects the current allocation of resources to processes.
  - The state consists of the two vectors, Resource and Available, and the two matrices, Claim and Allocation.
  - A state of the system is called safe if the system can allocate all the resources requested by all the processes without entering into deadlock.
  - If the system cannot fulfill the request of all processes then the state of the system is called unsafe.

Banker's Algorithm (contd…)

- Banker's algorithm is a deadlock avoidance and resource allocation algorithm.
- When a process makes a request for a set of resources, assume that the request is granted, update the system state accordingly, and then determine if the result is a safe state.
- If so, grant the request and, if not, block the process until it is safe to grant the request.

Banker's Algorithm – Example 1

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

Initial State

Banker's Algorithm – Example 1

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
6	2	3

Available vector V

P2 runs to completion

Banker's Algorithm – Example 1

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
7	2	3

Available vector V

P1 runs to completion

Banker's Algorithm – Example 1

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
9	3	4

Available vector V

P3 runs to completion, similarly P4 will also run to completion.

It is clear that the given state is safe state.

Banker's Algorithm – Example 2

Given the initial state, assume that P1 has an additional unit of R1 and R3.
Determine the state of the system.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
1	1	2

Available vector V

Initial State

If we assume that the request is granted, the state will be as shown in next slide.

Banker's Algorithm – Example 2

P1 requests one additional unit of R1 and R3

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

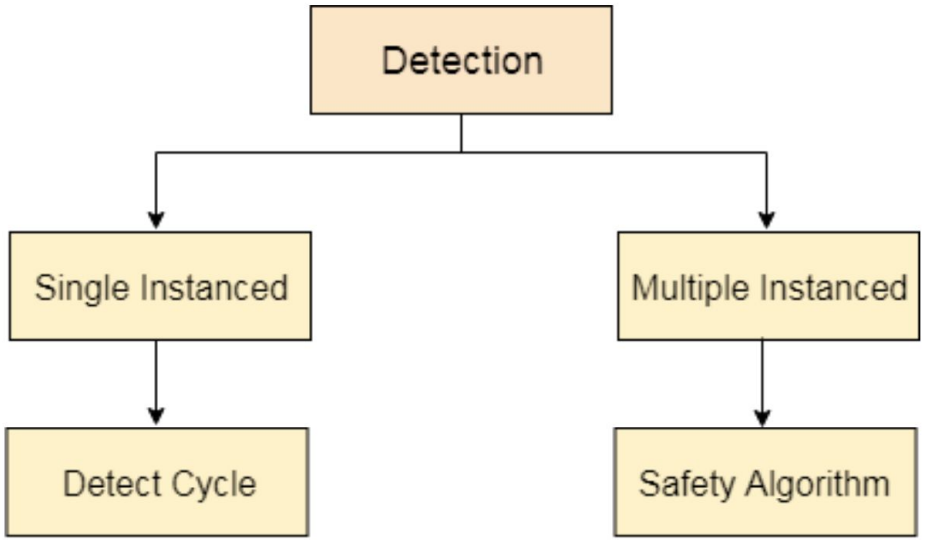
Is this a safe state? The answer is no, because each process will need at least one additional unit of R1, and there are none available.

Thus, on the basis of deadlock avoidance, the request by P1 should be denied and P1 should be blocked.

Deadlock Detection and Recovery

- In this approach, the operating system does not apply any mechanism to avoid or prevent the deadlocks.
- Therefore, the system considers that the deadlock will definitely occur.
- In order to get rid of deadlocks, the operating system periodically checks the system for any deadlock.
- In case, it finds any of the deadlock, then, the operating system will recover the system using some recovery techniques.
- The main task of the operating system is detecting the deadlocks. The operating system can detect the deadlocks with the help of Resource Allocation Graph.

Deadlock Detection and Recovery (contd..)



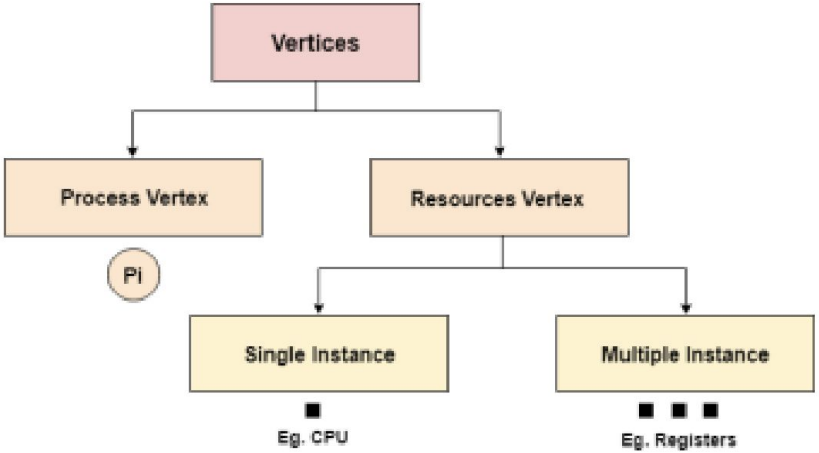
Resource-Allocation Graph

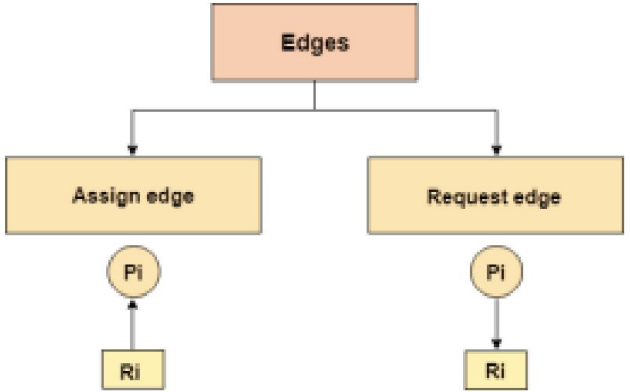
- Deadlocks can be described more precisely in terms of a directed graph, called a system **Resource-Allocation Graph**:
  - This graph consists of a set of vertices **V** and a set of edges **E**.
  - The set of vertices V is partitioned into two different types of nodes:
    - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the active processes in the system.
    - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system.
  - A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$ **(request edge)**; it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.
  - A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$ **(assignment edge)**; it signifies that an instance of resource type R_j has been allocated to process P_i .

Resource-Allocation Graph (contd…)

- Each process P_i is represented as a circle and each resource type R_j is represented as a rectangle.
  - Resource type R_j may have more than one instance, each instance is represented as a dot within the rectangle.
  - A request edge points to only the rectangle R_j, whereas an assignment edge must also designate one of the dots in the rectangle.
- When process P_i requests an instance of resource type R_j, a request edge is inserted in the resource-allocation graph.
- When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge.
- When the process no longer needs access to the resource, it releases the resource; as a result, the assignment edge is deleted.

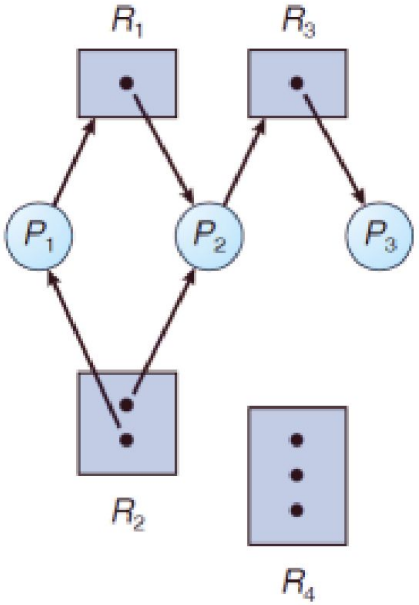
Resource-Allocation Graph (contd…)





Resource-Allocation Graph: Example

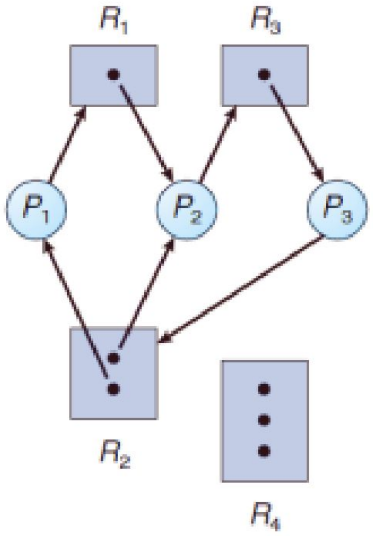
- P = {P1, P2, P3}
- R = {R1, R2, R3, R4}
- E = {P1 → R1, P2 → R3, R1 → P2, R2 → P2, R2 → P1, R3 → P3}



Deadlock Detection using RAG

- If the graph contains no cycles, then no process in the system is deadlocked.
- If the graph does contain a cycle, then a deadlock may exist.
- If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- Each process involved in the cycle is deadlocked.
- In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

Deadlock Detection using RAG: Example

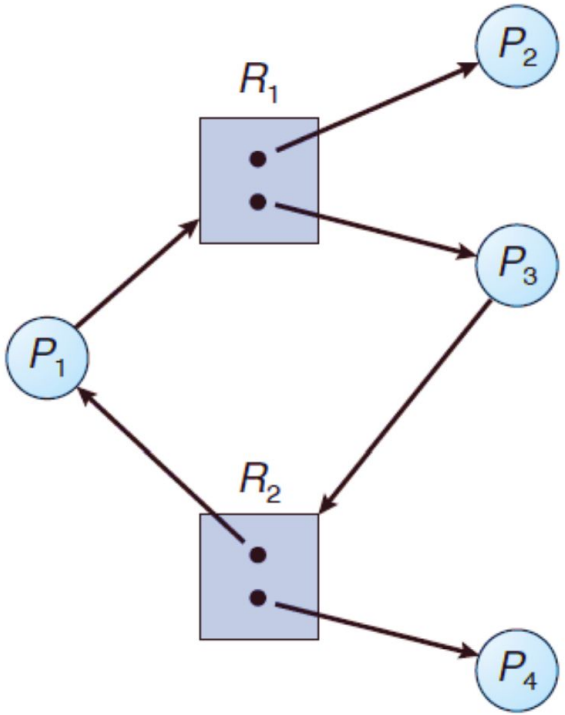


- **Cycle 1:** $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- **Cycle 2:** $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$
- Processes P_1, P_2, and P_3 are deadlocked.

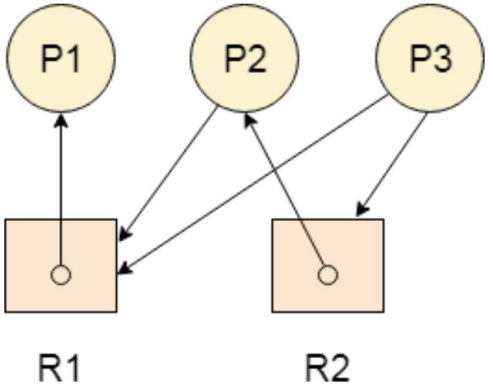
Deadlock Detection using RAG: Example

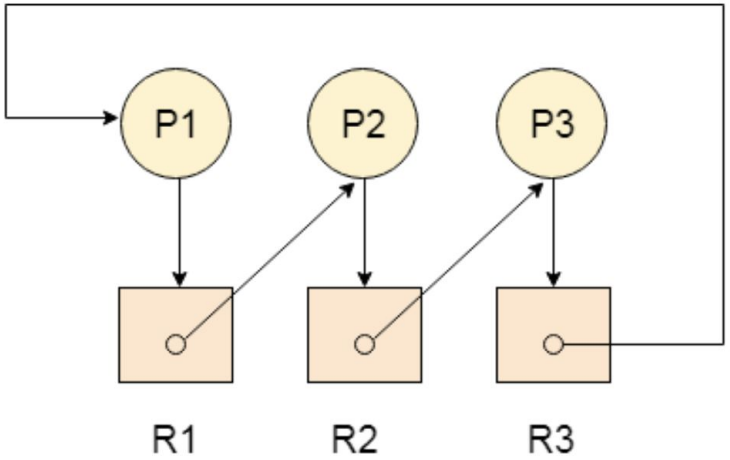
- Cycle 1: $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- There is no deadlock.
- The process P_4 may release its instance of resource type R_2. That resource can then be allocated to P_3, breaking the cycle.

In summary, if a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state. If there is a cycle, then the system may or may not be in a deadlocked state.

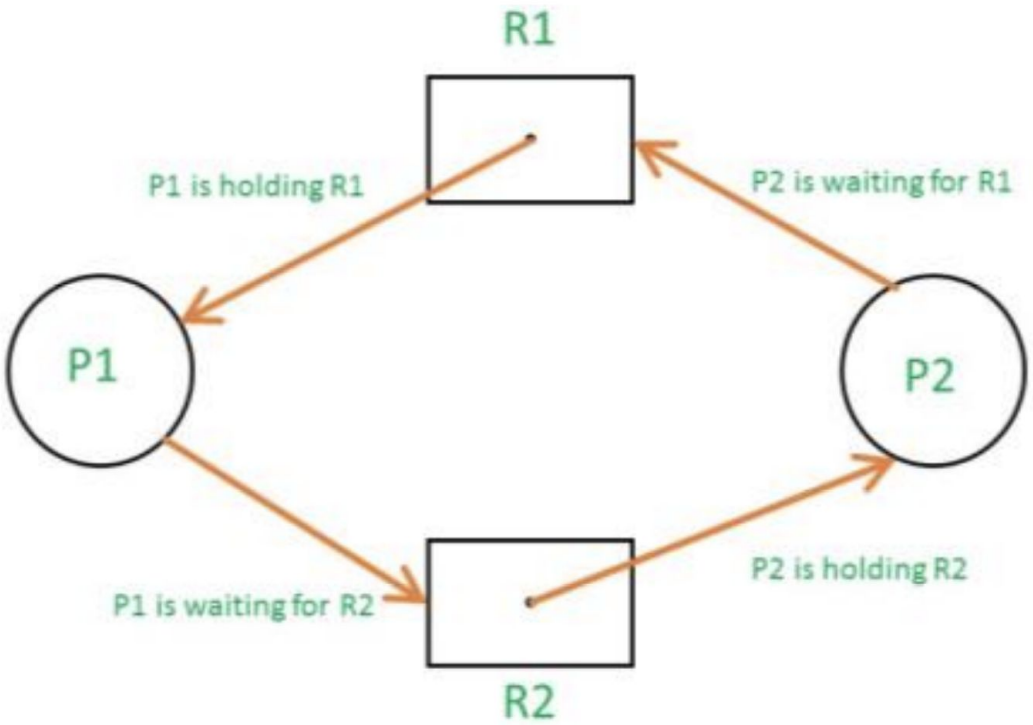


Deadlock Detection using RAG: Example

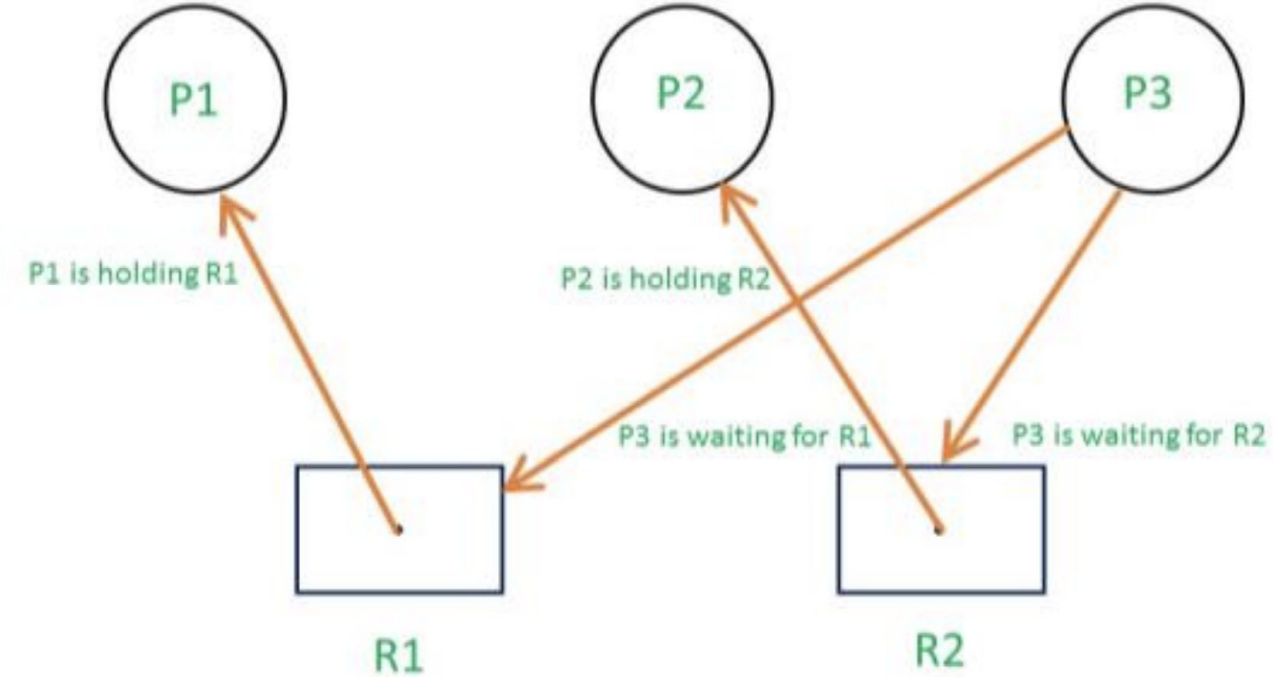




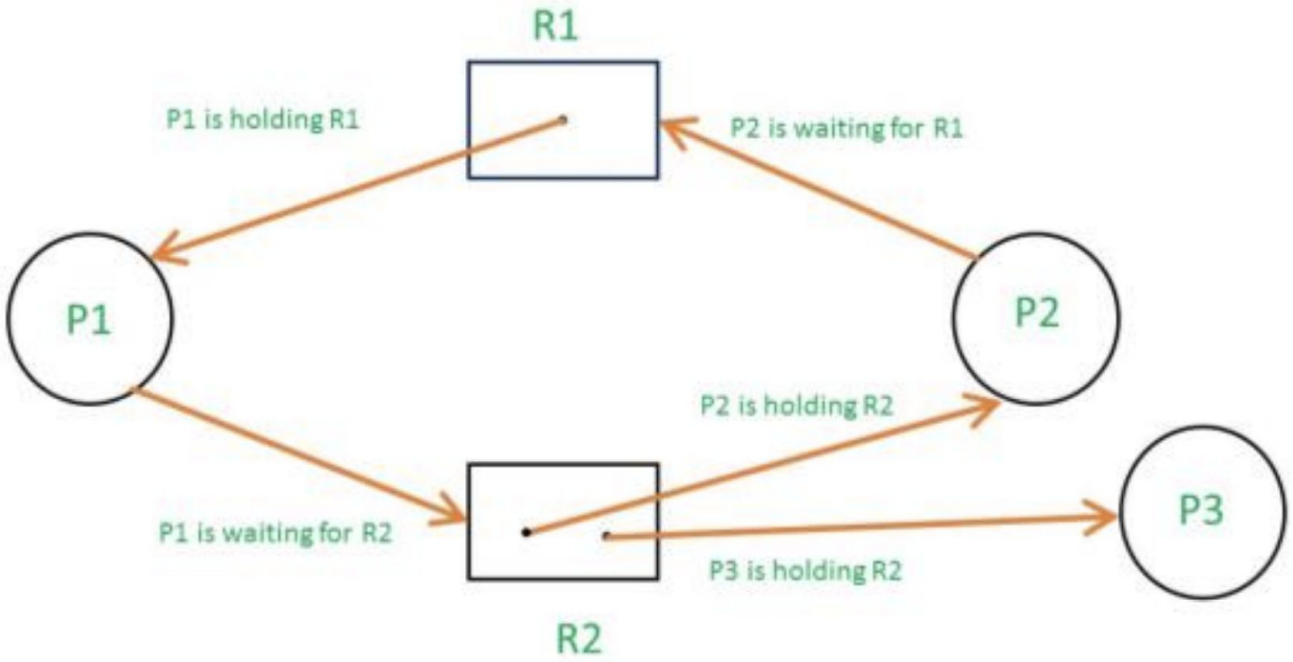
Deadlock Detection using RAG: Example



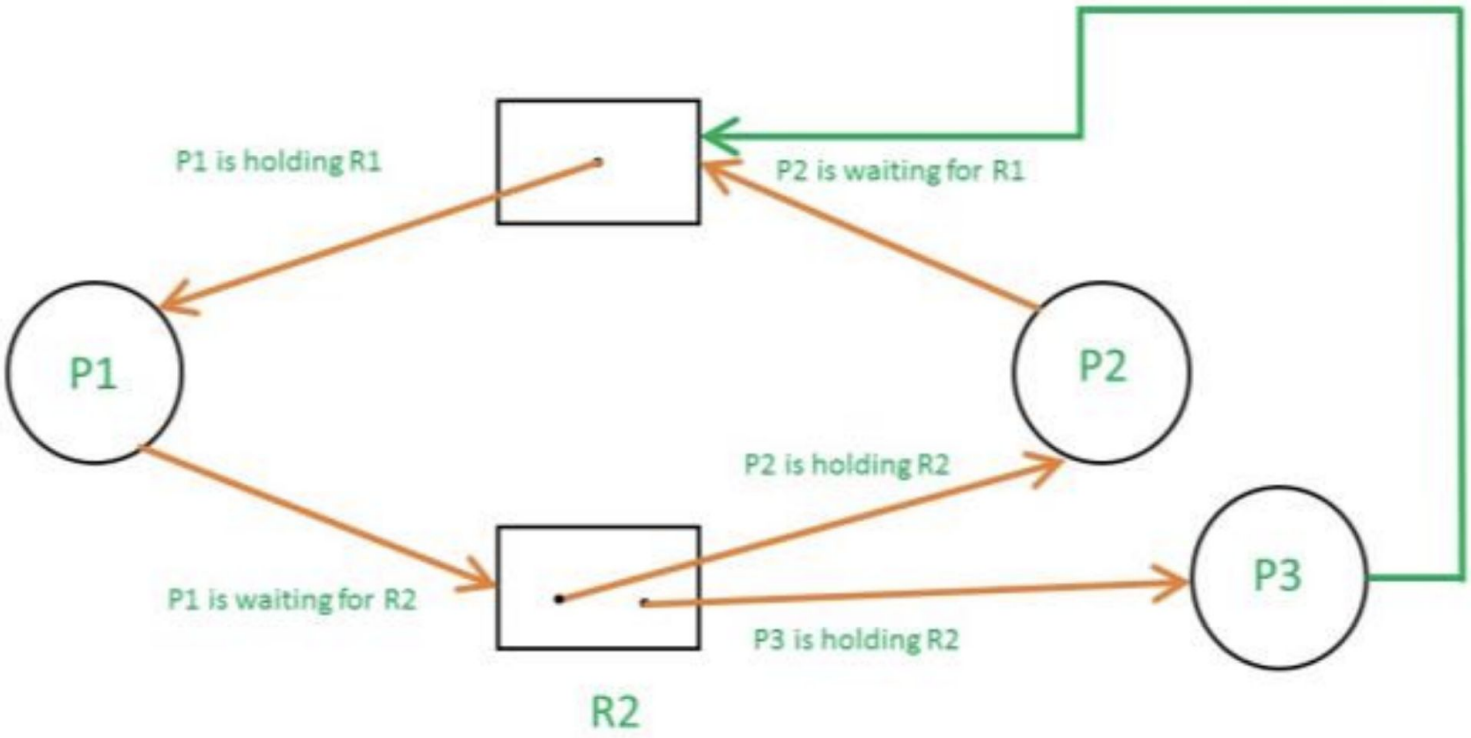
Deadlock Detection using RAG: Example



Deadlock Detection using RAG: Example



Deadlock Detection using RAG: Example



Deadlock Detection for Several Instance of Resource

- Let, an Allocation Matrix, a Request Matrix Q, and an Available Vector
- The algorithm proceeds by marking processes that are not deadlocked. Initially, all processes are unmarked. Then the following steps are performed:

1. Mark each process that has a row in the Allocation matrix of all zeros.
2. Initialize a temporary vector **W** to equal the Available vector.
3. Find an index i such that process i is currently unmarked and the ith row of **Q** is less than or equal to **W**. That is, $Q_{ik} \leq W_k$, for $1 \leq k \leq m$. If no such row is found, terminate the algorithm.
4. If such a row is found, mark process i and add the corresponding row of the allocation matrix to **W**. That is, set $W_k = W_k + A_{ik}$, for $1 \leq k \leq m$. Return to step 3.

- A deadlock exists if and only if there are unmarked processes at the end of the algorithm. Each unmarked process is deadlocked.

Deadlock Detection - Example

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

- Mark P4, because P4 has no allocated resources.
- Set W = (0 0 0 0 1).
- The request of process P3 is less than or equal to W, so mark P3 and set W = W + (0 0 0 1 0) = (0 0 0 1 1).
- No other unmarked process has a row in Q that is less than or equal to W. Therefore, terminate the algorithm.
- The algorithm concludes with P1 and P2 unmarked, indicating that these processes are deadlocked.

Deadlock Recovery

- To recover deadlock, the operating system examines either resources or processes.
- For Process:
  - Kill a Process: In this approach, kill the process due to which deadlock occurred. But the selection of the process to kill is a tough task. In this, the operating system mainly kills that process, which does not work more till now.
  - Kill all Process: Kill all the processes is not a suitable approach. We can use this approach when the problem becomes critical. By killing all the processes, the system efficiency will be decreased, and we have to execute all the processes further from the start.

Deadlock Recovery (contd…)

- For Resources:
  - Preempt the Resource: In this, we take the resource from one process to the process that needs it to finish its execution, and after the execution is completed, the process soon releases the resource. In this, the resource selection is difficult, and the snatching of the resource is also difficult.
  - Rollback to a Safe State: To enter into the deadlock, the system goes through several states. In this, the operating system can easily roll back the system to the earlier safe state. To do so, we require to implement checkpoints at every state. At the time when we detect deadlock, then we need to rollback every allocation so that we can enter into the earlier safe state.

Background: Memory

- Main memory and the registers built into the processor itself are the only storage that the CPU can access directly.
- There are machine instructions that take memory addresses as arguments, but none take disk addresses.
- Therefore, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices.
- If the data are not in memory, they must be moved there before the CPU can operate on them.
- Registers that are built into the CPU are generally accessible within one clock cycle.
- Most CPUs can decode instructions and perform simple operations on register contents at the rate of one or more operations per clock tick.
  - The same cannot be said of main memory, which is accessed via a transaction on the memory bus.

Background: Memory (contd…)

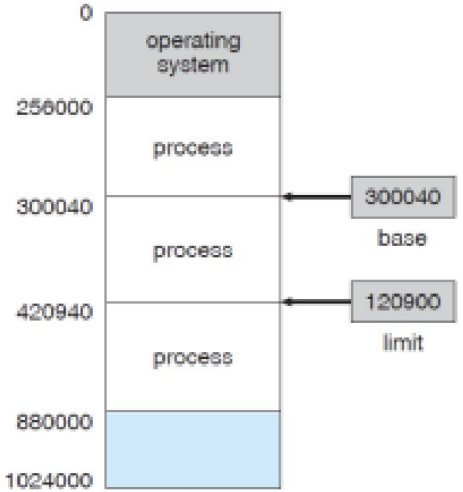
- Completing a memory access may take many clock cycles.
- In such cases, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.
- This situation is intolerable because of the frequency of memory accesses.
- The remedy is to add fast memory between the CPU and main memory.
- A memory buffer used to accommodate a speed differential, is called cache.

Basic Hardware

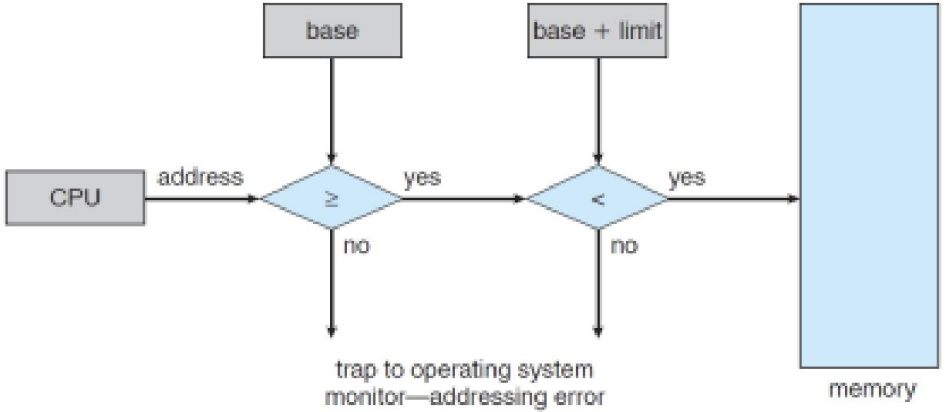
- There are several hardware-level approaches to protect the operating system from access by user processes and, in addition, to protect user processes from one another.
- To make sure that each process has a separate memory space, the protection is provided by using two registers: **Base Register** and **Limit Register**.
  - For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).
  - Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers.
  - Any attempt by a program executing in user mode, to access operating-system memory or other users' memory, results in a trap to the operating system, which treats the attempt as **a fatal error** (an error that causes a program to terminate without any warning or saving its state).
    - This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.

Hardware Address Protection

- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.



Base and Limit Register defining a logical address space



Hardware address protection with Base and Limit Registers

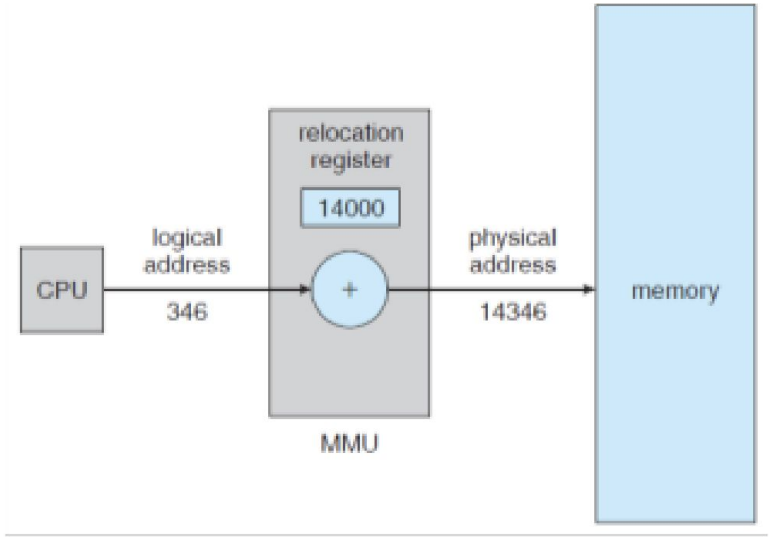
Logical versus Physical Address Space

- An address generated by the CPU is commonly referred to as a logical address.
  - The logical address is virtual address as it does not exist physically, therefore, it is also known as virtual address.
  - This address is used as a reference to access the physical memory location by CPU.
- An address seen by the memory unit (that is, the one loaded into the memory-address register of the memory) is referred to as a physical address.
  - Physical address identifies a physical location of required data in a memory.
- The compile-time and load-time address-binding methods generate identical logical and physical addresses.
- The execution-time address-binding scheme results in differing logical and physical addresses.

Logical and Physical Address Space

- The set of all logical addresses generated for a program's perspective is a logical address space.
- The set of all physical addresses corresponding to these logical addresses is a physical address space.
- The run-time mapping from virtual to physical addresses is done by a hardware device called the Memory-Management Unit (MMU).

Mapping from Virtual to Physical Addresses

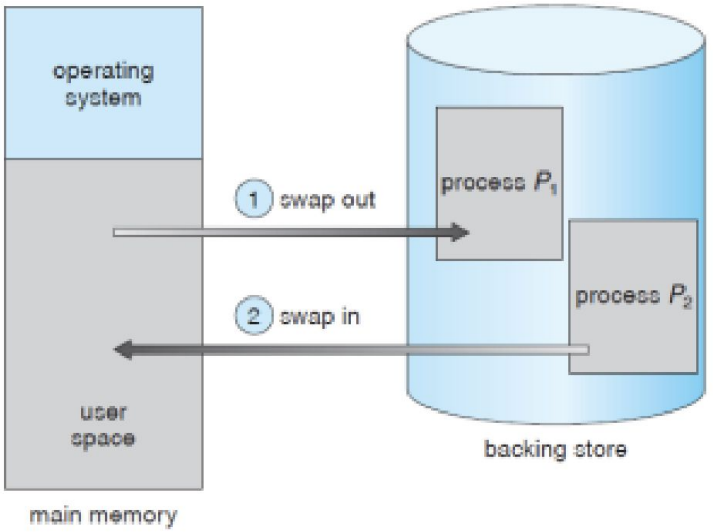


Example: Simple MMU scheme (a generalization of the base-register scheme)

- CPU generates logical address 346.
- MMU generates relocation register (base register) 14000.
- In memory, the physical address is 14346 (346+14000).

Swapping

- **A process must be in memory to be executed.**
- A process can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.
- **Example**: Assume a multiprogramming environment with a round-robin CPU-scheduling algorithm. When a quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into the memory space that has been freed.



Swapping (contd…)

- A variant of swapping policy is used for priority-based scheduling algorithms.
- If a higher-priority process arrives and wants service, the memory manager can swap out the lower-priority process and then load and execute the higher-priority process.
- When the higher-priority process finishes, the lower-priority process can be swapped back in and continued.
- This variant of swapping is sometimes called roll out, roll in.
- Normally, a process that is swapped out will be swapped back into the same memory space it occupied previously.
- This restriction is dictated by the method of address binding. If binding is done at assembly or load time, then the process cannot be easily moved to a different location. If execution-time binding is being used, then a process can be swapped into a different memory space, because the physical addresses are computed during execution time.

Swapping (contd…)

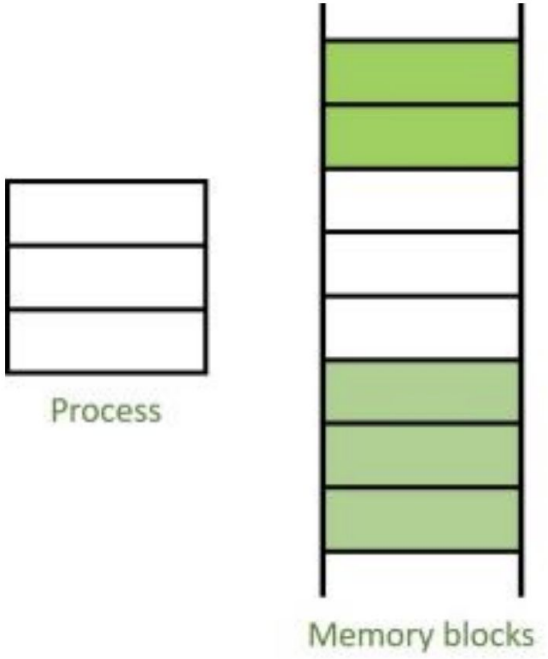
- Swapping requires a backing store (a fast disk, large enough to accommodate copies of all memory images, and it must provide direct access to these memory images).
- System maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run.
- Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.
- The dispatcher checks to see whether the next process in the queue is in memory.
- If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired.

Memory Management Techniques

- There are two techniques for memory management:
  - Contiguous Memory Allocation
  - Non-Contiguous Memory Allocation
- In Contiguous Memory Allocation, the process must be loaded entirely in main-memory at contiguous locations.
- In Non-Contiguous Memory Allocation, the process is loaded in several memory blocks at different memory locations in the memory.

Contiguous Memory Allocation

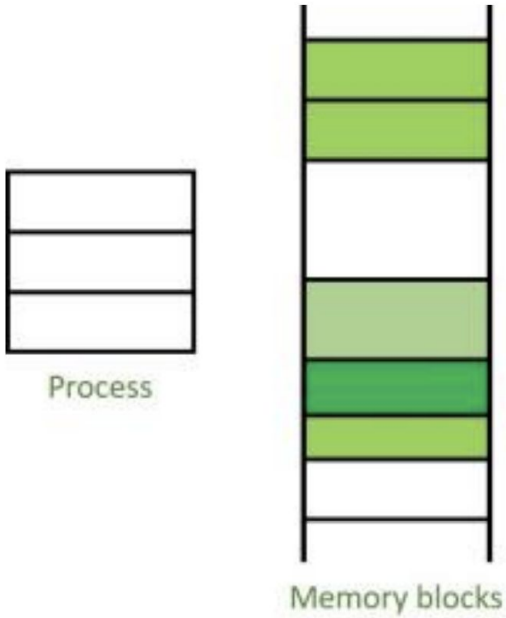
- A single contiguous section/part of memory is allocated to a process or file needing it.



Contiguous Memory Allocation

Non-Contiguous Memory Allocation

- The process is loaded in several memory blocks at different memory locations in the memory.



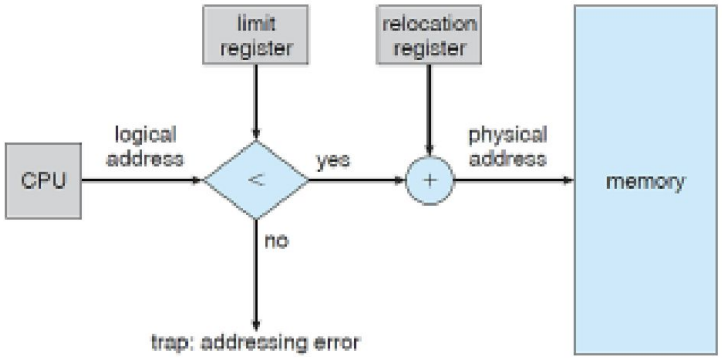
Noncontiguous Memory Allocation

Contiguous Memory Allocation

- In contiguous memory allocation, each process is contained in a single contiguous block of memory.
- Discuss -
  - Memory Mapping and Protection
  - Partitioning
  - Allocation Policies
  - Performance Parameters
    - Fragmentation
    - Maximum Process Size
    - Degree of Multiprogramming
    - Allocation Policy

Memory Mapping and Protection

- These features can be provided by using a relocation register together with a limit register.
  - The relocation register contains the value of the smallest physical address; the limit register contains the range of logical addresses (for example, relocation = 100040 and limit = 74600).



  - When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.
  - Every address generated by a CPU is checked against these registers, we can protect both the operating system and other users' programs and data from being modified by this running process

Memory Allocation

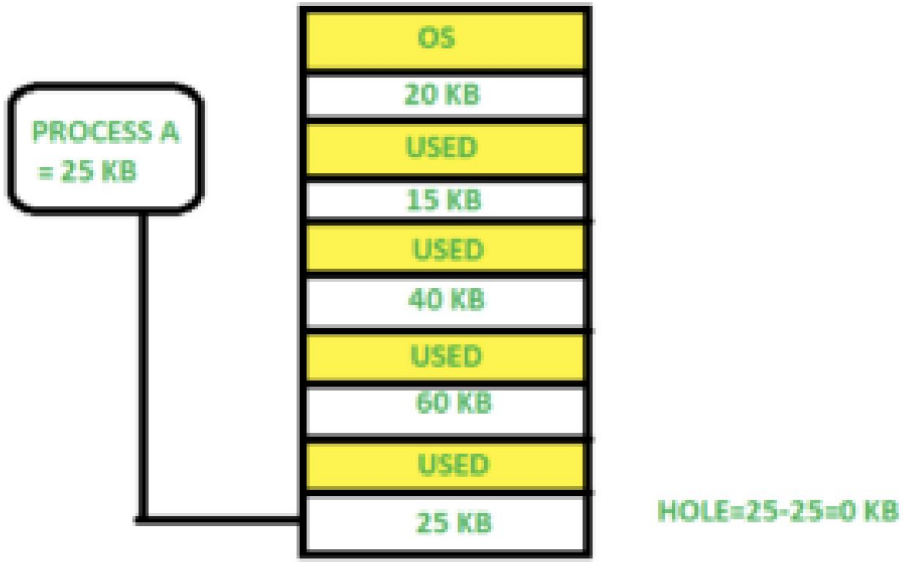
- Dividing memory into several partitions is one of the simplest methods for allocating memory.
  - Each partition may contain exactly one process.
  - When a partition is free, a process is selected from the input queue and is loaded into the free partition.
  - When the process terminates, the partition becomes available for another process.
- Partitioning can be done in two ways:
  - Fixed (Static) Partitioning
  - Variable (Dynamic) Partitioning

Memory Allocation Policies

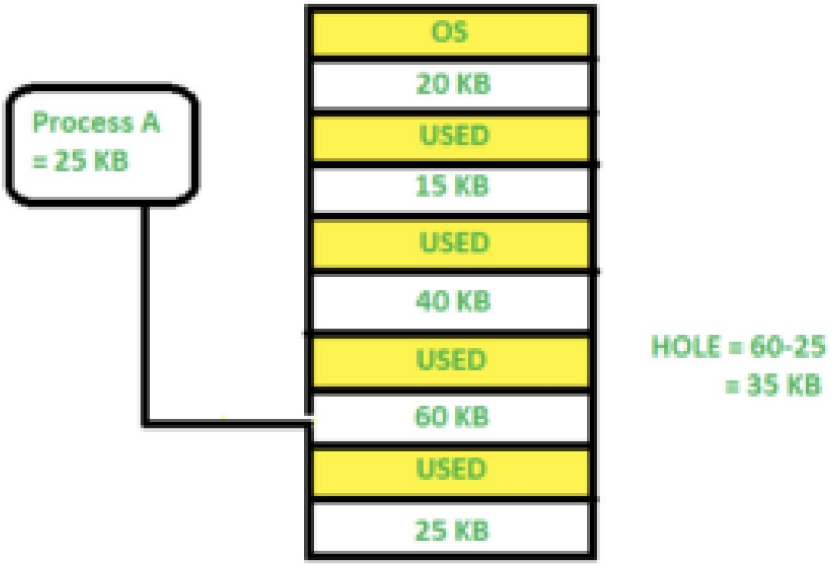
- The memory blocks available comprise a set of holes of various sizes scattered throughout memory. When a process arrives and needs memory, the system can search an appropriate hole with following policies:
  - First-Fit - Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended.
  - Best-Fit - Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
  - Worst-Fit - Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole.
  - Next-Fit - It is similar to the first fit but it will search for the first sufficient partition from the last allocation point.

Memory Allocation Policies: First Fit

Memory Allocation Policies: Best-Fit



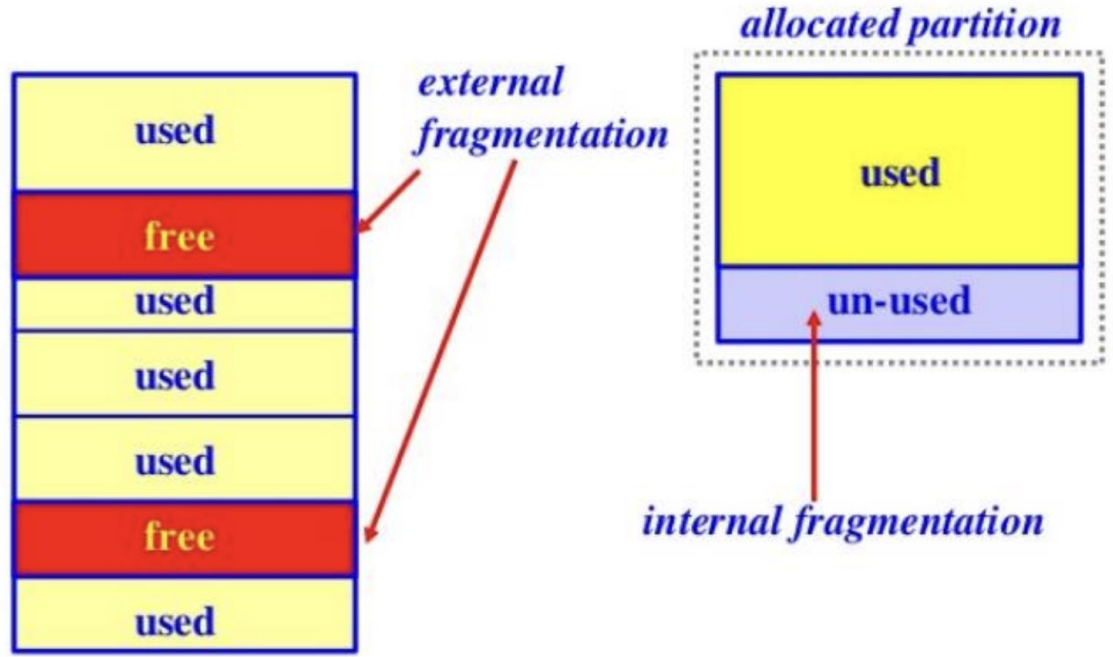
Memory Allocation Policies: Worst-Fit



Fragmentation

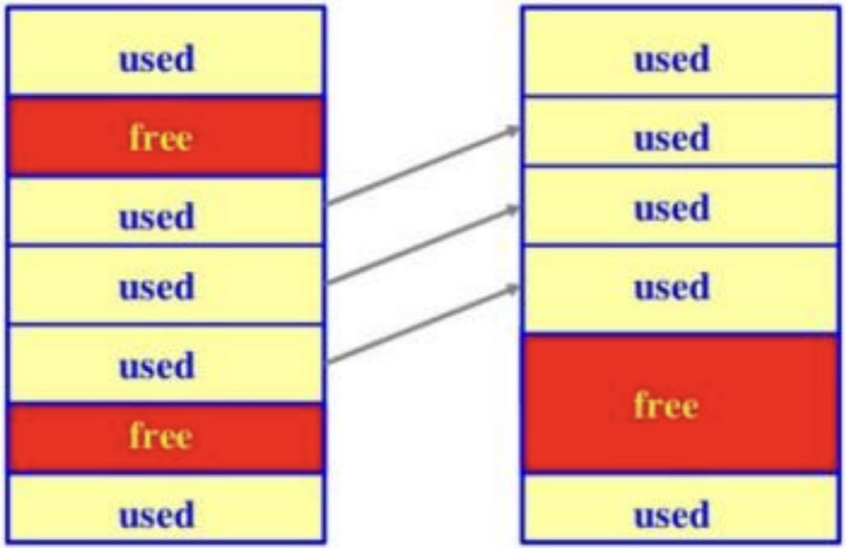
- Internal Fragmentation
  - Memory block allocated is slightly larger than request memory, therefore, some portion of memory is left unused, as it cannot be used by another process.
  - The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.
- External Fragmentation
  - Total memory space exists to satisfy a request, but it is not contiguous.
  - The external fragmentation may be reduced by compaction (also known as defragmentation) – shuffle memory contents to place all free memory together in one large block.
  - **Compaction is possible only if relocation is dynamic, and is done at execution time.**

Fragmentation



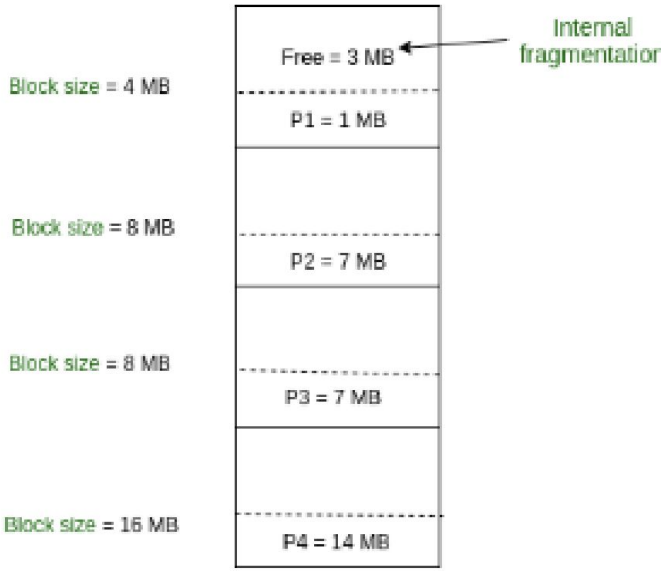
Compaction

- If processes are relocatable, the used memory blocked may be moved together to make a larger free memory block.



Fixed Partitioning

- In this partitioning, number of partitions in memory are fixed but size of each partition may or may not be same.
- As it is contiguous allocation, hence no spanning is allowed.
- Here, partition are made before execution or during system configure.
- Sum of internal fragmentation in every block = (4-1)+(8-7)+(8-7)+(16-14) = 3 + 1 + 1 + 2 = 7MB.
- Suppose a process P5 of size 7MB comes. But, this process cannot be accommodated inspite of available free space because of contiguous allocation. Hence, 7MB becomes part of External Fragmentation.

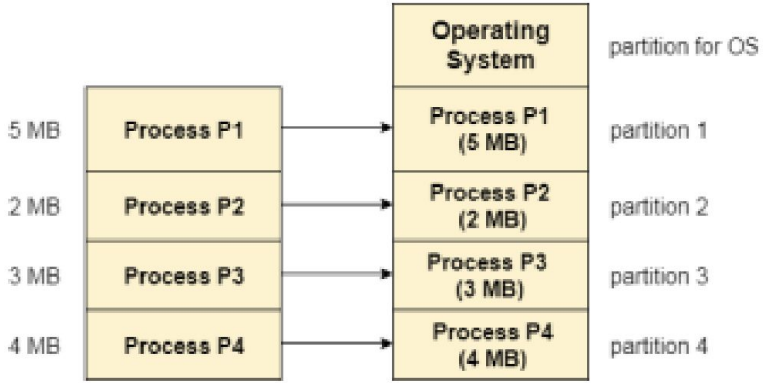


Advantages/Disadvantages of Fixed Partitioning

- Advantages
  - Easy to implement
  - Little OS overhead
- Disadvantages
  - Internal fragmentation
  - External fragmentation
  - Limit process size - Process of size greater than size of partition in Main Memory cannot be accommodated.
  - Limitation on degree of multiprogramming
- Best Allocation Policy
  - Best-Fit

Variable Partitioning

- This partitioning tries to overcome the problems caused by fixed partitioning.
- In this technique, the partition size is not declared initially.
- It is declared at the time of process loading.
- The partition size varies according to the need of the process so that the internal fragmentation can be avoided.
- The size of each partition will be equal to the size of the process.

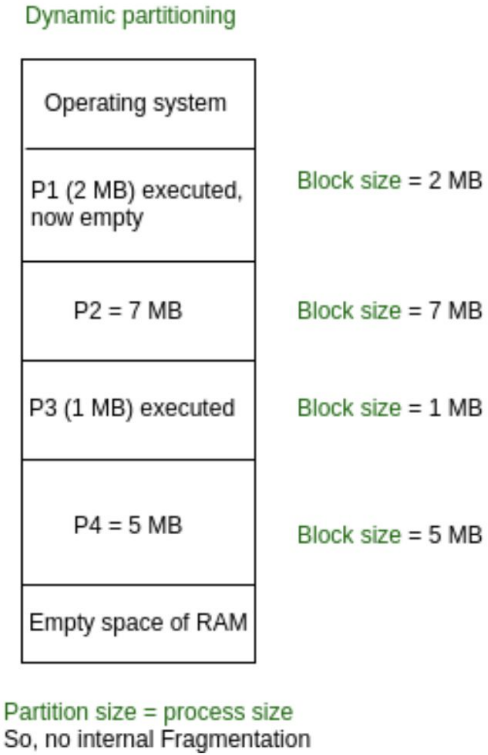


Advantages/Disadvantages of Variable Partitioning

- Advantages
  - No internal fragmentation
  - No limitation on the size of the process
  - Degree of multiprogramming is dynamic
- Disadvantages
  - External fragmentation
- Best Allocation Policy
  - Worst-Fit

Advantages/Disadvantages of Variable Partitioning

- External Fragmentation
  - Suppose, Process P1 (2MB) and process P3 (1MB) completed their execution. Hence two spaces are left, i.e. 2MB and 1MB.
  - Let's suppose process P5 of size 3MB comes. The empty space in memory cannot be allocated as no spanning is allowed in contiguous allocation.
  - The rule says that process must be contiguously present in main memory to get executed.
  - Hence, variable partitioning may result in external fragmentation.



Paging

- Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous.
- Paging avoids external fragmentation and the need for compaction.
- Paging in its various forms is used in most operating systems.
- Implementation of paging involves dividing the process into blocks of the same size called pages which are mapped to same size blocks on physical memory called frames.
  - Physical Memory Blocks --> Frames
  - Logical Memory Blocks --> Pages

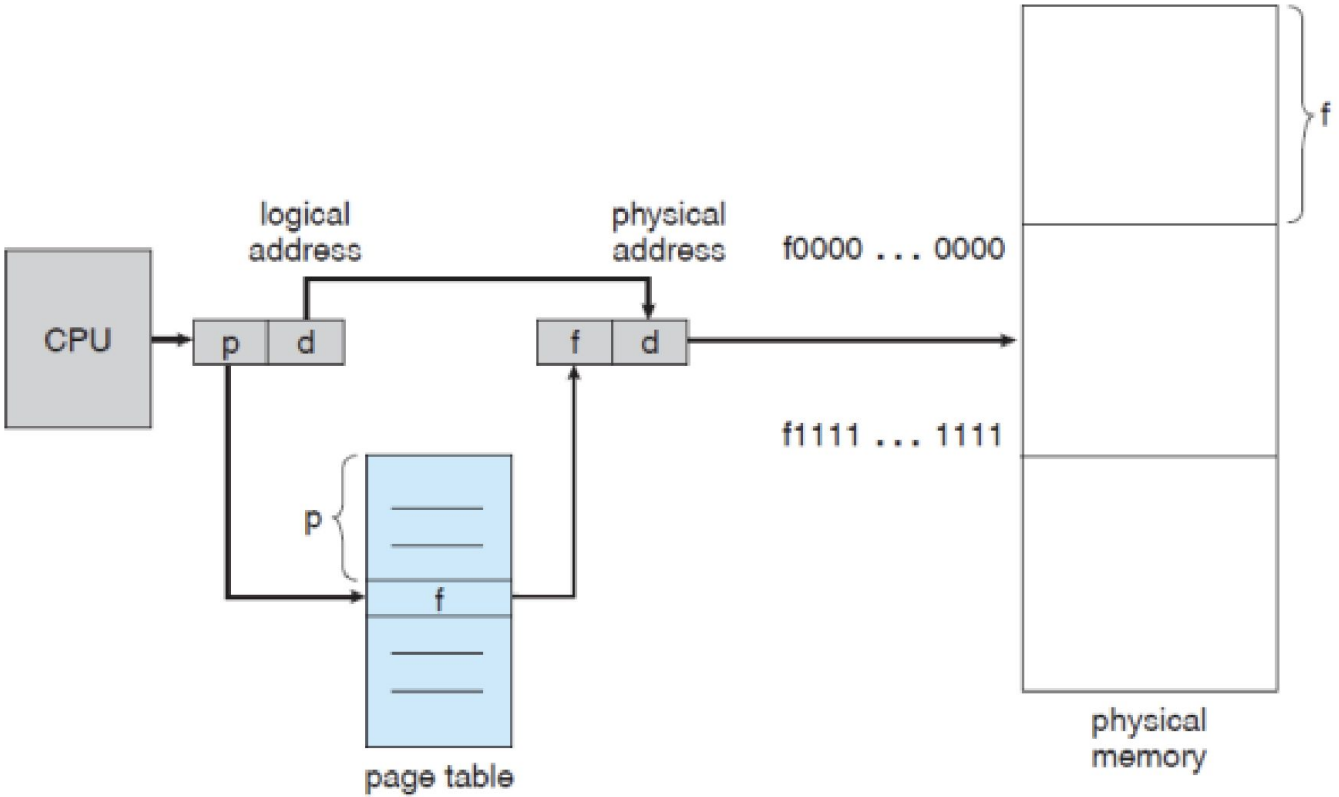
Paging (contd…)

- In every non-contiguous memory management technique, we need to consider the following points:
  - Organization of Logical Address Space (LAS)
  - Organization of Physical Address Space (PAS)
  - Organization of Memory Management Unit (MMU)
  - Translation Algorithm

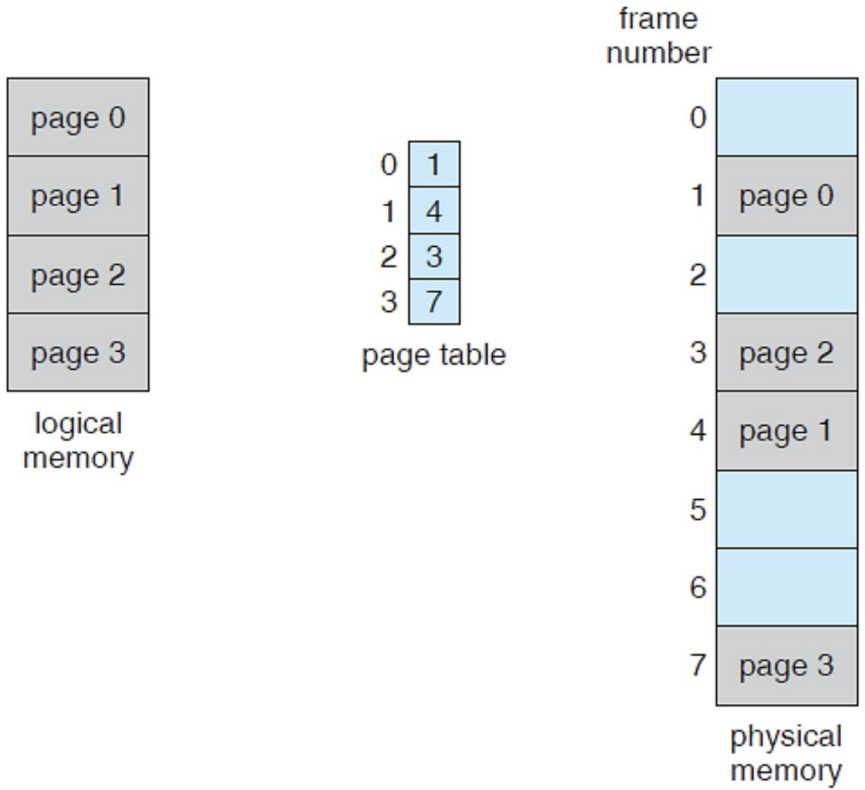
Paging (contd…)

- **Address generated by CPU is divided into:**
  - Page number (p): Number of bits required to represent the pages in Logical Address Space.
  - Page offset (d): Number of bits required to represent particular word in a page or word number of a page.
- **Physical Address is divided into:**
  - Frame number (f): Number of bits required to represent the frame of Physical Address Space.
  - Frame offset (d): Number of bits required to represent particular word in a frame or word number of a frame.

Paging (contd…)



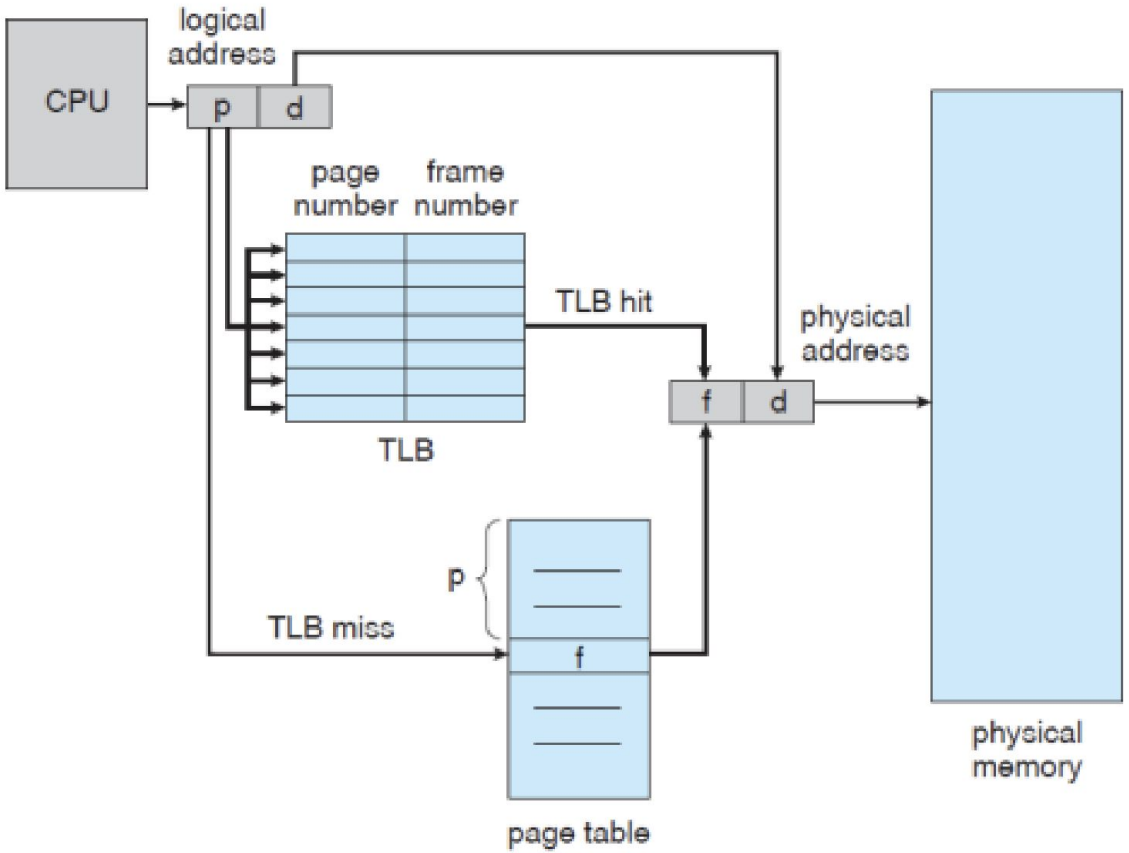
Paging (contd…)



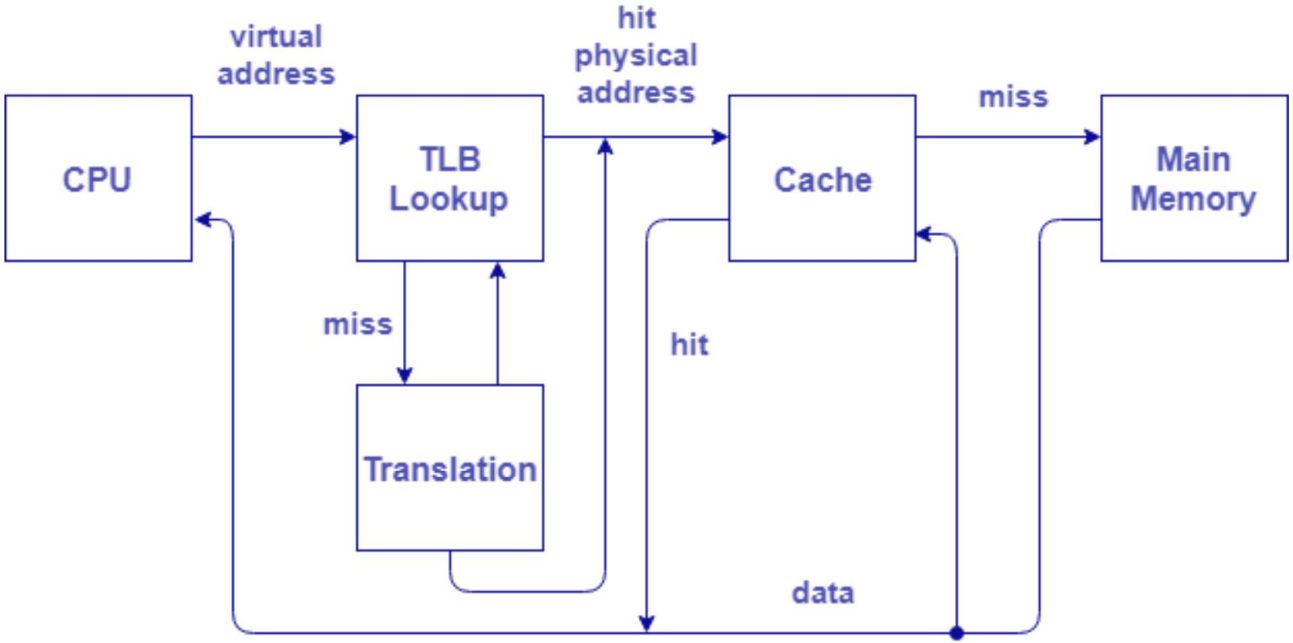
Using TLB with Page Table

- The TLB contains only a few of the page-table entries.
- When a logical address is generated by the CPU, its page number is presented to the TLB.
  - If the page number is found, its frame number is immediately available and is used to access memory.
  - If the page number is not in the TLB **(known as a TLB miss)**, a memory reference to the page table must be made. In addition, we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.
  - If the TLB is already full of entries, the operating system must select one for replacement. Replacement policies range from least recently used (LRU) to random. Furthermore, some TLBs allow certain entries to be wired down, meaning that they cannot be removed from the TLB. Typically, TLB entries for kernel code are wired down.

Using TLB with Page Table



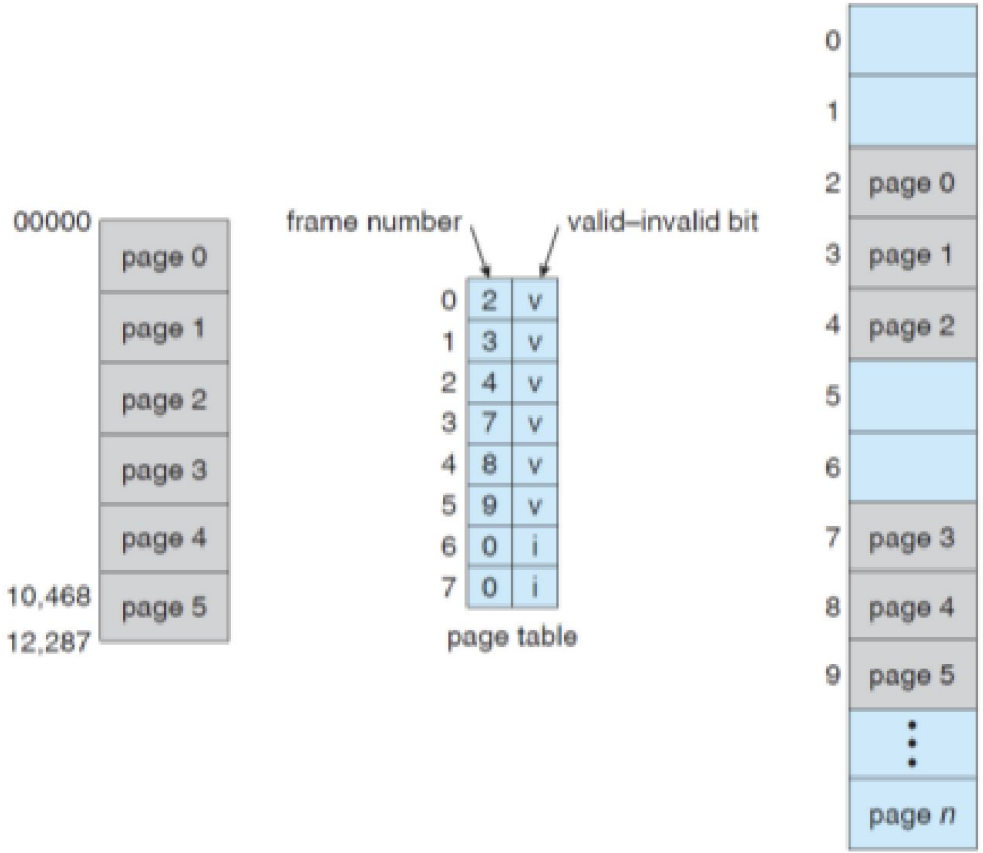
CPU Cache and TLB



Protection in Paging: Valid–Invalid Bit

- One additional bit is generally attached to each entry in the page table: a valid–invalid bit.
  - When this bit is set to "valid," the associated page is in the process's logical address space and is thus a legal (or valid) page.
  - When the bit is set to "invalid," the page is not in the process's logical address space.
  - The OS sets this bit for each page to allow or disallow access to the page.

Protection in Paging: Valid–Invalid Bit

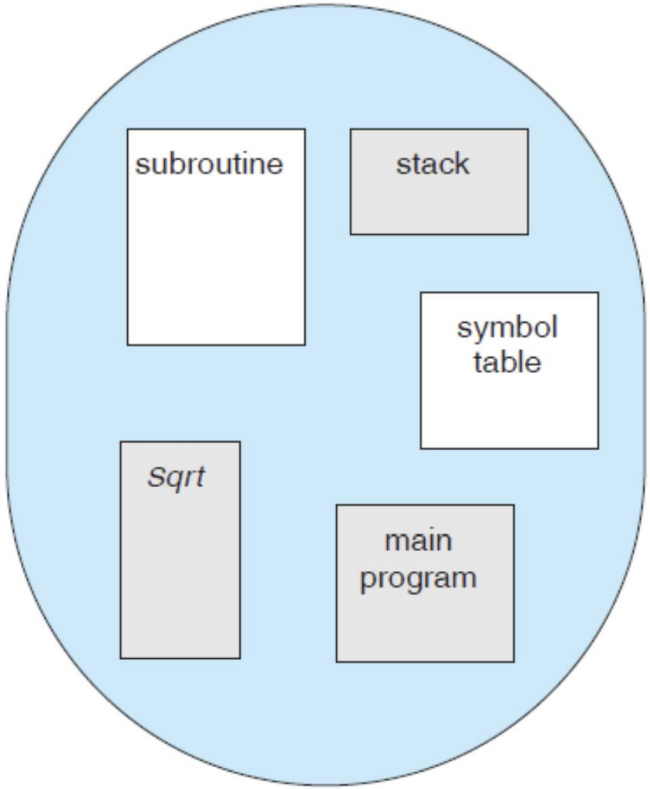


Limitations of Paging

- Internal Fragmentation
  - If the memory requirements of a process do not happen to coincide with page boundaries, the last frame allocated may not be completely full.
  - **Example**: If page size is 2048 bytes, a process of 72,766 bytes will need 35 pages plus 1,086 bytes. It will be allocated 36 frames, resulting in internal fragmentation of 2,048 − 1,086 = 962 bytes.
  - If process size is independent of page size, we expect internal fragmentation to average one-half page per process.
  - This consideration suggests that small page sizes are desirable. However, overhead is involved in each page-table entry, and this overhead is reduced as the size of the pages increases. Also, disk I/O is more efficient when the amount of data being transferred is larger.
- Larger Access Time
- Memory Required for Page Table

Segmentation

- In Paging, the user's view of memory is not the same as the actual physical memory.
- Users do not think of memory as a linear array of bytes, some containing instructions and others containing data.
- Rather, users prefer to view memory as a collection of variable-sized segments (methods, stack, etc.), with no necessary ordering among segments.



User's View of a Program

Segmentation (contd…)

- Segmentation is a memory-management scheme that supports user's view of memory.
- A logical address space is a collection of segments.
- Each segment has a name and a length.
- The addresses specify both the segment number and the offset within the segment.
  - Elements within a segment are identified by their offset from the beginning of the segment: the first statement of the program, the seventh stack frame entry in the stack, the fifth instruction of the Sqrt(), and so on.

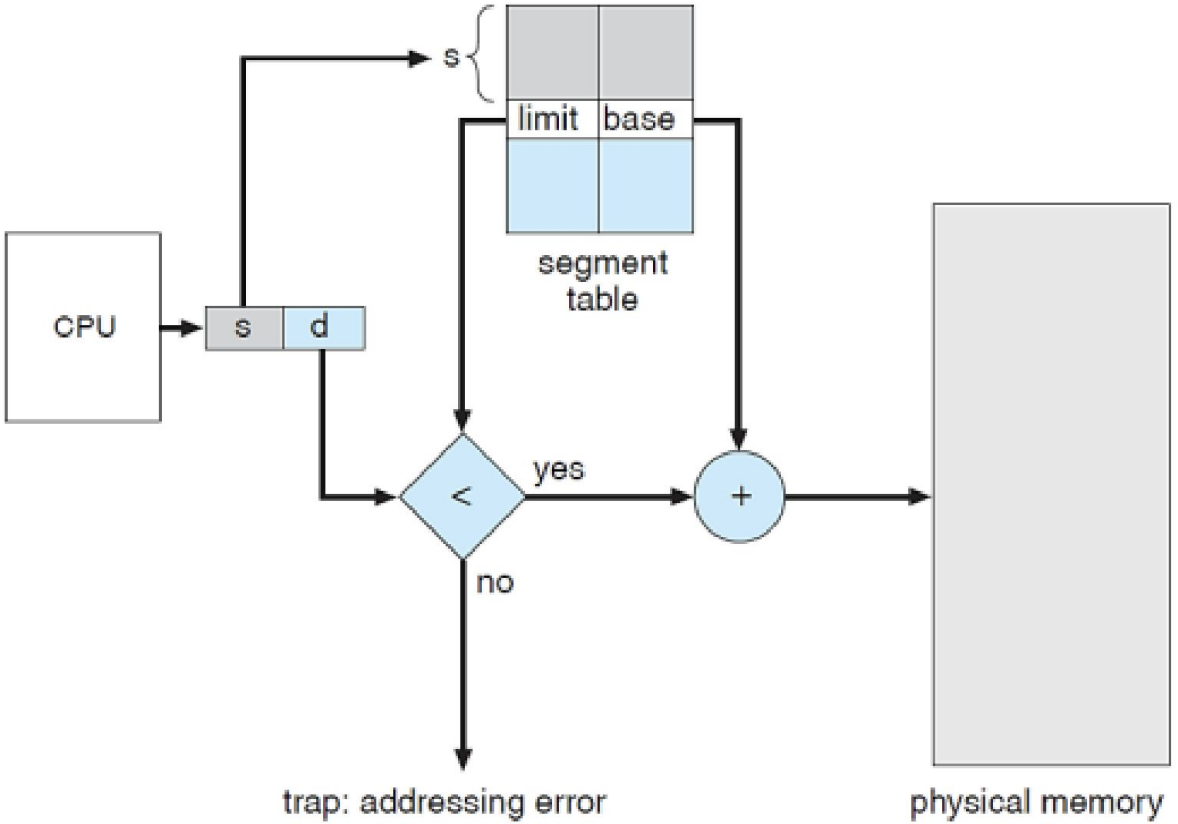
Segmentation (contd…)

- While compiling, the compiler automatically constructs segments reflecting the input program.
- A ‘C’ compiler might create separate segments for the following:
  - The code
  - Global variables
  - The heap, from which memory is allocated
  - The stacks used by each thread
  - The standard C library
  - Libraries that are linked in during compile time
- The loader takes all these segments and assign them segment numbers.

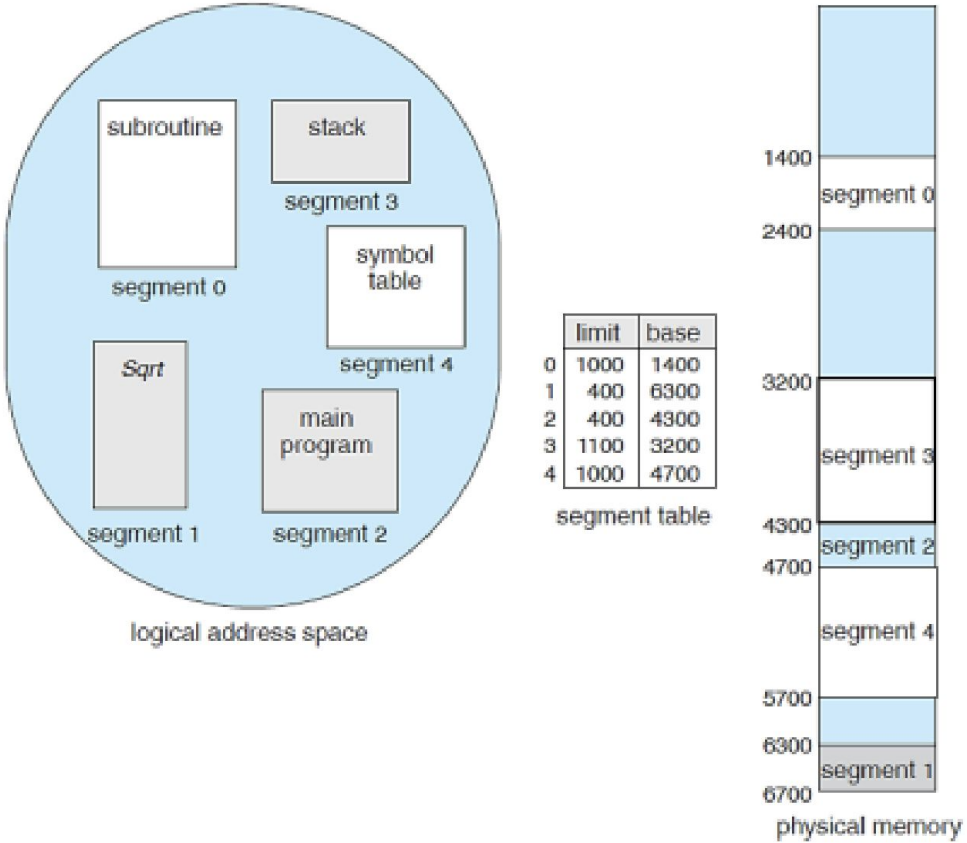
Segmentation (contd…)

- Each entry in the segment table has a segment base and a segment limit.
  - The segment base contains the starting physical address where the segment resides in memory.
  - The segment limit specifies the length of the segment.
- A logical address consists of two parts:
  - A segment number **(s)**, and
  - An offset into that segment **(d)**
- The segment number is used as an index to the segment table.
- The offset **d** of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system.

Segmentation (contd…)



Segmentation (contd…)



Limitations of Segmentation

- **External Fragmentation**
  - To deal with external fragmentation, the Segmentation can be combined with Paging, also known as **Segmented Paging**.

Virtual Memory

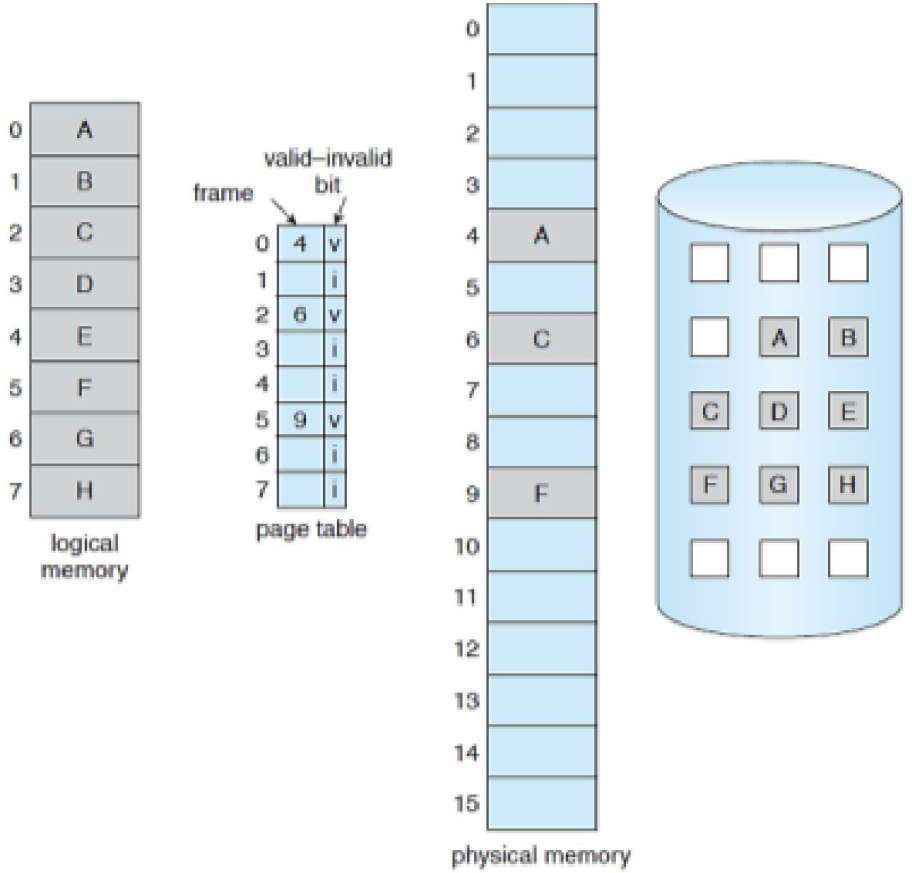
- Virtual memory is a technique that allows the execution of processes that are not completely in memory.
- One major advantage of this scheme is that programs can be larger than physical memory.
- Virtual memory gives an illusion to the user/programmer that huge amount memory is available for executing its process.
- Virtual memory can be implemented through demand paging.

Demand Paging

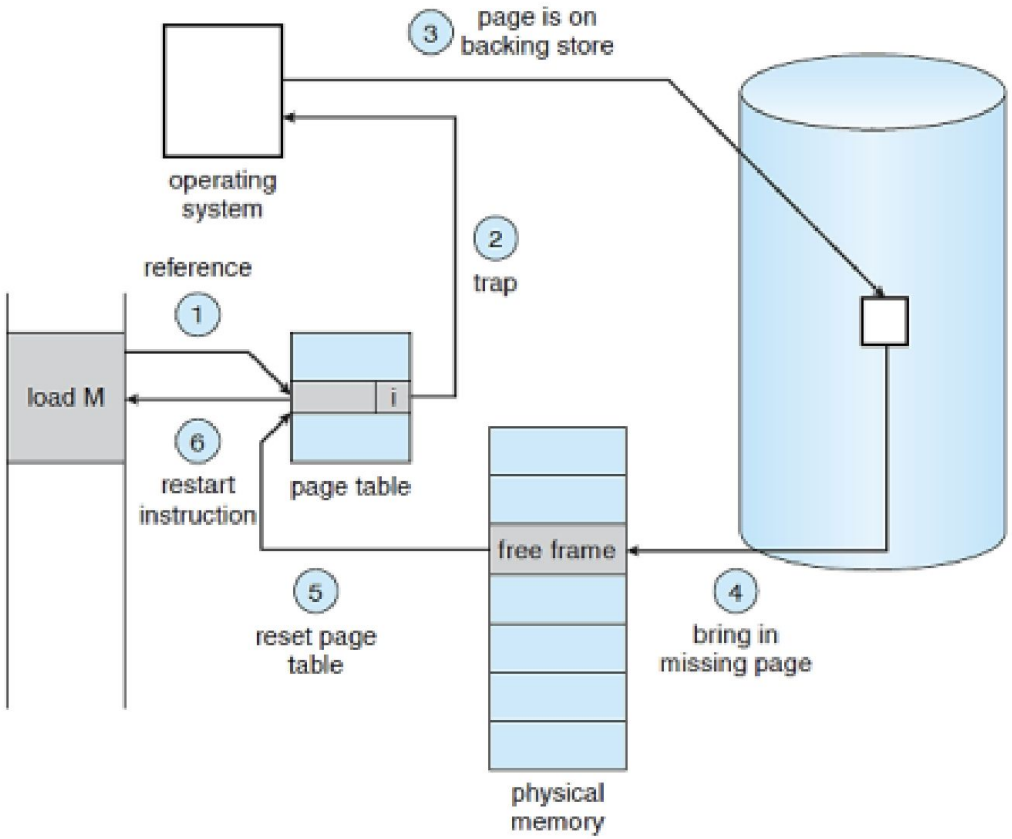
- Demand Paging
  - Pure Demand Paging - Never bring a page into memory until it is required
  - Pre-fetched Demand Paging
- A demand-paging system is similar to a paging system with swapping.
- Types of Pages
  - Modified Page or Dirty Page
  - Clean Page
- Page Fault Service Time

Page Table when some Pages are not in Main Memory

- To distinguish between the pages that are in memory and the pages that are on the disk, we need some form of hardware support.
- The valid–invalid bit scheme can be used for this purpose.
- When this bit is set to "valid," the associated page is in memory.
- If the bit is set to "invalid", the page is currently on the disk.



Steps in Handling a Page Fault



Performance of Virtual Memory

- Let
  - Main Memory Access Time (MMAT) = M
  - Page Fault Rate (PFR) = P
  - Page Fault Service Time (PFST) = S
  - Effective Memory Access Time (EMAT) = P*S+(1-P)*M
- If MMAT = 200 microsecond, PFR = 25%, PFST = 1 millisecond, then what is EMAT (in Microsecond)?

EMAT = 0.25*1000 + 0.75*200 = 250 + 150 = 400 Microsecond

Page Replacement

1) Find the location of the desired page on the disk.

2) Find a free frame:

   a) If there is a free frame, use it.

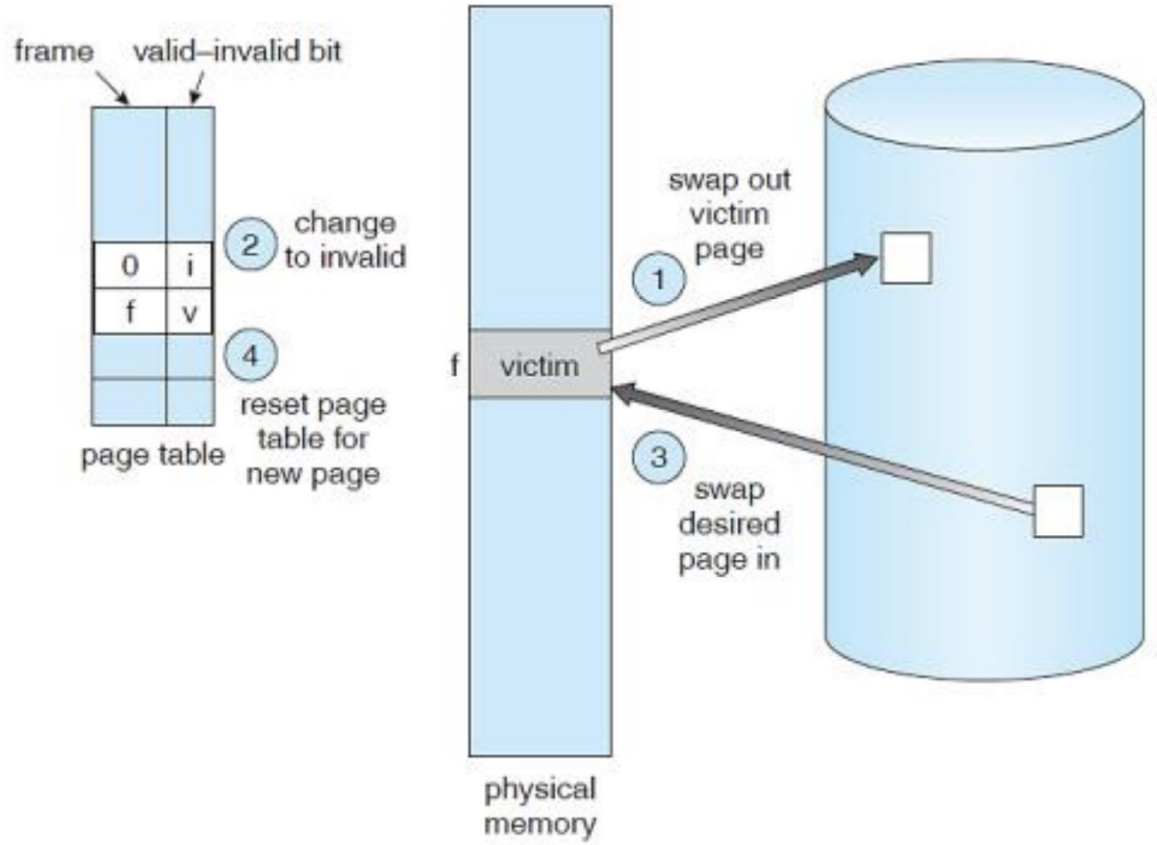
   b) If there is no free frame, use a page-replacement algorithm to select a victim frame.

   c) Write the victim frame to the disk (If dirty); change the page and frame tables accordingly.

3) Bring the desired page into the (newly) freed frame; change the page and frame tables.

4) Continue the process by restarting the instruction that caused the trap.

Page Replacement (contd…)



Page Replacement Algorithms

- First-Come, First-Out
- Optimal Page Replacement
- Least Recently Used

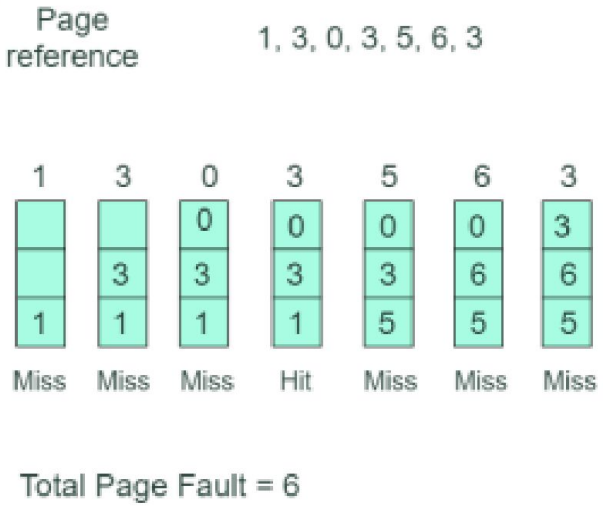
We can evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults.

- The string of memory references is called a reference string.

Page Replacement Algorithm: First-Come, First-Out

- We can create a FIFO queue to hold all pages in memory. We replace the page at the head of the queue. When a page is brought into memory, we insert it at the tail of the queue.

Q. Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find number of page faults.



Page Replacement Algorithm: First-Come, First-Out

Q. Consider page reference string 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4 with 3 page frames. Find number of page faults.

Q. Consider page reference string 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4 with 4 page frames. Find number of page faults.

Belady's Anomaly – For some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

Page Replacement Algorithm: Optimal Page Replacement

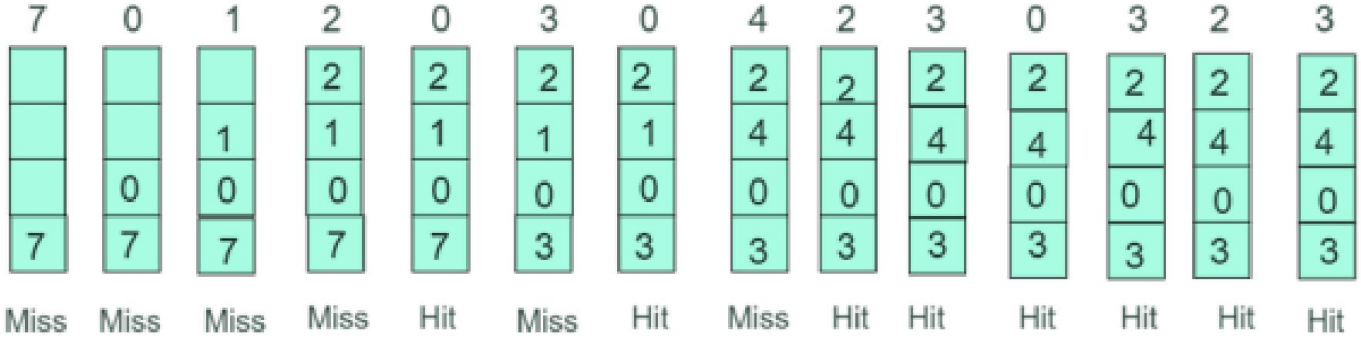
Replace the page that will not be used for the longest period of time.

- Use of this page-replacement algorithm guarantees the lowest possible page fault rate for a fixed number of frames.
- Unfortunately, the optimal page-replacement algorithm is difficult to implement, because it requires future knowledge of the reference string.
- The optimal algorithm is used mainly for comparison studies.

Page Replacement Algorithm: Optimal Page Replacement

Q. Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, with 4 page frame. Find number of page fault using optimal page replacement algorithm.

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3 No. of Page frame - 4



Total Page Fault = 6

Page Replacement Algorithm: Least-Recently-Used (LRU)

Replace the page that has not been used for the longest period of time.

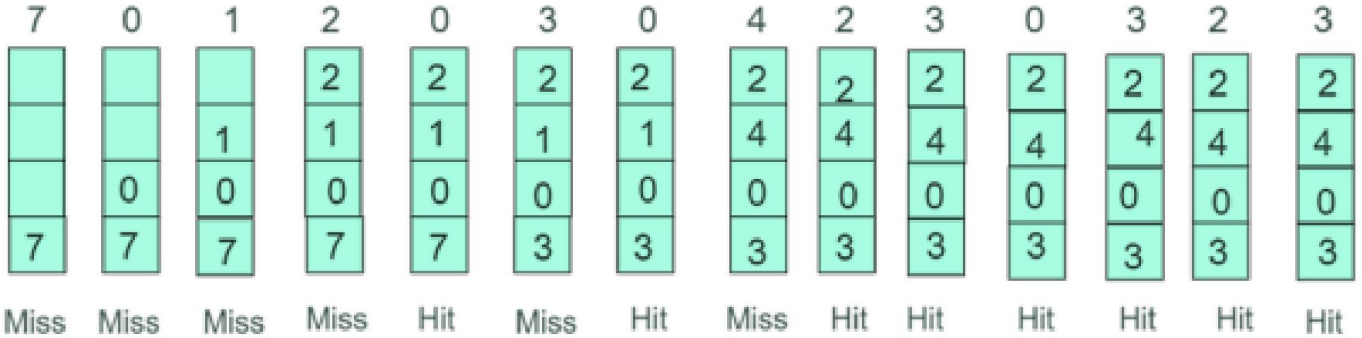
- LRU replacement associates with each page the time of that page's last use.
- When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.
- This strategy can be considered as the optimal page-replacement algorithm looking backward in time, rather than forward.

- The LRU policy is often used as a page-replacement algorithm and is considered to be good.

Page Replacement Algorithm: LRU Example

Q. Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 4 page frames. Find number of page faults using least-recently-used algorithm.

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3 No. of Page frame - 4



Total Page Fault = 6

Allocation of Frames

- How do we allocate the fixed amount of free memory among the various processes?
  - The minimum number of frames per process is defined by the architecture.
  - The maximum number is defined by the amount of available physical memory.

Allocation of Frames: Equal Allocation

- The easiest way to split m frames among n processes is to give everyone an equal share, m/n frames.
- For instance, if there are 93 frames and five processes, each process will get 18 frames. The three leftover frames can be used as a free-frame buffer pool.

Allocation of Frames: Proportional Allocation

- Consider a system with a 1-KB frame size.
  - If a small student process of 10 KB and an interactive database of 127 KB are the only two processes running in a system with 62 free frames, it does not make much sense to give each process 31 frames.
  - The student process does not need more than 10 frames, so the other 21 are, strictly speaking, wasted.
- To solve this problem, we can use proportional allocation, in which we allocate available memory to each process according to its size.

Allocation of Frames: Global vs. Local Allocation

- With multiple processes competing for frames, we can classify page-replacement algorithms into two broad categories:
  - Global Replacement
    - It allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; that is, one process can take a frame from another.
    - We may allow high-priority processes to select frames from low-priority processes for replacement.
  - Local Replacement
    - It allows that that each process select from only its own set of allocated frames.
    - With a local replacement strategy, the number of frames allocated to a process does not change.

Thrashing

- If the process does not have the number of frames it needs to support pages in active use, it will quickly page-fault.
- At this point, it must replace some page.
- However, since all its pages are in active use, it must replace a page that will be needed again right away.
- Consequently, it quickly faults again, and again, and again, replacing pages that it must bring back in immediately.
- This high paging activity is called **thrashing**.
- A process is thrashing if it is spending more time in paging than executing.

Thrashing